



LINUX

AUTOMATION

COOKBOOK



SCRIPTS



AUTOMATION



SYSTEMS



OPERATIONS



RELIABILITY

1. WHY AUTOMATE?

- ✓ Reduce manual work and human errors
- ✓ Improve speed and consistency
- ✓ Enable repeatable, reliable operations
- ✓ Handle routine and large-scale tasks
- ✓ Free engineers to focus on higher value work

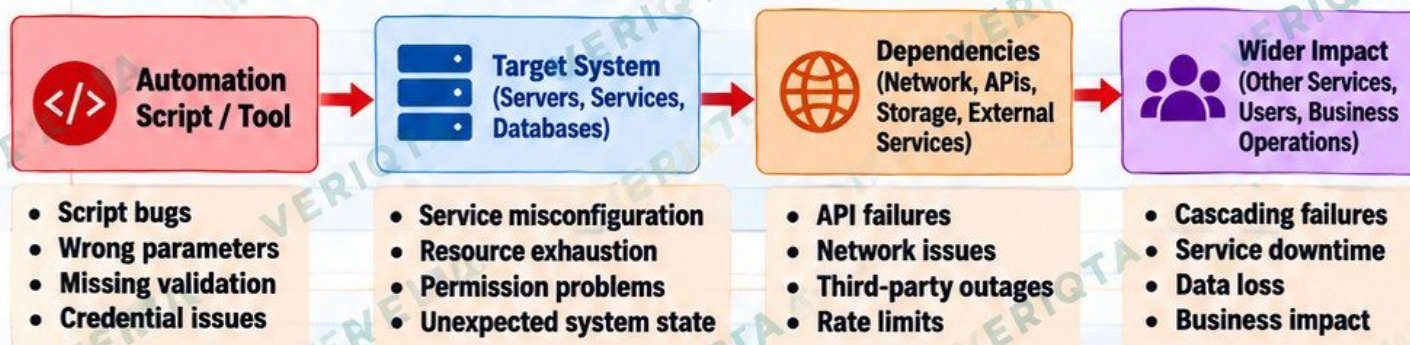
2. WHAT SHOULD NOT BE AUTOMATED?

- ✗ High-risk, irreversible actions
e.g. deleting production data
- ✗ Tasks with no clear success criteria
- ✗ Unstable or frequently changing processes
- ✗ Complex decisions requiring human judgment
- ✗ Actions without proper monitoring or rollback
- ✗ Anything you cannot safely test first

3. WHAT SHOULD BE AUTOMATED?

- ✓ System provisioning and configuration
- ✓ Software deployment and updates
- ✓ Health checks and remediation (with guardrails)
- ✓ Backups and disaster recovery tasks
- ✓ Log rotation, cleanup and routine maintenance
- ✓ Monitoring, reporting and alerting setup
- ✓ Repeatable incident response actions

4. AUTOMATION FAILURE DOMAINS



5. HUMAN vs MACHINE RESPONSIBILITY

 HUMAN RESPONSIBILITY	 MACHINE RESPONSIBILITY
- Design the automation strategy - Define guardrails and policies - Review and approve high-risk changes - Handle complex or ambiguous situations - Monitor outcomes and make improvements - Take ownership of failures	- Execute well-defined, repeatable tasks - Follow rules and guardrails - Perform validations and checks - Operate at scale and high speed - Collect evidence and generate reports - Trigger alerts on anomalies

6. DESIGNING FOR REPEATABILITY

-  **Idempotent**
Safe to run multiple times without causing harm
-  **Predictable**
Same input → same output
-  **Validated**
Check preconditions and verify results
-  **Observable**
Log actions, failures and outcomes
-  **Safe by default**
Include dry-run, rollback and clear boundaries



SENIOR ENGINEER TIP

Automate only what you can monitor, validate, and safely recover. If you cannot measure the outcome or roll it back, it is not ready for production.



REAL WORLD LESSON

Many major outages have been caused by automated scripts without proper validation, rollback or scope controls. Treat your automation with the same engineering discipline as any production system.

02 Idempotency: Safe to Run Twice

1. WHAT IS IDEMPOTENCY?

- ✓ An operation is idempotent when running it multiple times produces the same result as running it once.
- ✓ It is safe to run again without causing unwanted changes or side effects.
- ✓ Idempotency is essential for reliable, repeatable and production-grade automation.

2. DESIRED STATE vs IMPERATIVE EXECUTION

DESIRED STATE (Idempotent)

- ✓ Define the end state and enforce it.
- ✓ Check current state before making changes.
- ✓ Running again keeps the same state.

Example:

Ensure a package is installed.
Ensure a user exists.

IMPERATIVE EXECUTION (Non-idempotent)

- ✗ Execute actions blindly.
- ✗ May cause duplicate changes.
- ✗ Running again can break the system or create unwanted state.

Example:

Always add a user.
Always append to a file.

3. DETECT BEFORE CHANGING

- ✓ Always check the current state before making changes.
- ✓ Use commands that confirm existence or configuration.
- ✓ Change only when the desired state is not already met.
- ✓ This prevents unnecessary changes and avoids failures.

Check and install a package only if missing

```
if ! dpkg -s nginx >/dev/null 2>&1; then
    sudo apt update
    sudo apt install -y nginx
else
    echo "nginx is already installed"
fi
```

4. PARTIAL EXECUTION

- ✓ Automation can fail halfway due to network issues, permission errors or system interruptions.
- ✓ Design scripts to handle partial execution safely.
- ✓ Verify each step and make sure rerunning the script brings the system to the correct state.
- ✓ Use checkpoints, state validation and cleanup when needed.

Example: Create a user safely

```
if id "deploy" >/dev/null 2>&1; then
    echo "User deploy already exists"
else
    sudo useradd -m -s /bin/bash deploy
    echo "User deploy created"
fi
```

5. DESIGNING RERUNNABLE OPERATIONS

- 1 Use idempotent commands or state checks.
- 2 Validate inputs and preconditions.
- 3 Make operations safe to run multiple times.
- 4 Handle failures and partial execution.
- 5 Leave the system in a consistent state.
- 6 Log actions and decisions.
- 7 Test by running the script multiple times.

6. EXAMPLE: RERUNNABLE CONFIGURATION

```
CONFIG_FILE="/etc/nginx/nginx.conf"
EXPECTED_LINE="worker_processes auto;"

if grep -qxF "$EXPECTED_LINE" "$CONFIG_FILE"; then
    echo "Already configured"
else
    sudo sed -i "/^worker_processes/d" "$CONFIG_FILE"
    echo "$EXPECTED_LINE" | sudo tee -a "$CONFIG_FILE"
    echo "Configuration updated"
fi
```

7. KEY BENEFITS

- ✓ Prevents duplicate changes
- ✓ Reduces failures and manual fixes
- ✓ Safer automation in production
- ✓ Supports consistent, repeatable deployments
- ✓ Makes scripts reliable and easier to maintain



SENIOR ENGINEER TIP

Design every automation with the assumption that it can be interrupted and rerun. If it is not safe to run twice, it is not ready for production.



REAL WORLD LESSON

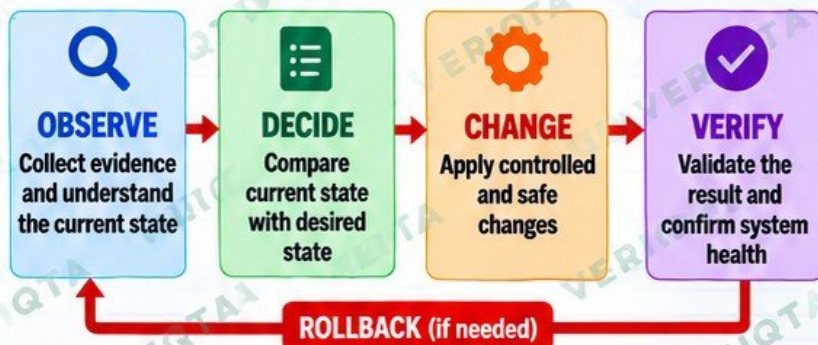
Many production incidents happen because scripts were not idempotent. A script that worked on a fresh server failed when rerun, creating duplicate users, broken configs or service downtime.

03 The Observe, Decide, Change, Verify Pattern

1. PATTERN OVERVIEW

- ✓ A safe and repeatable approach for production automation.
- ✓ Reduces risk and prevents unintended changes.
- ✓ Uses evidence to make decisions, not assumptions.
- ✓ Works for scripts, configuration changes and infrastructure automation.

2. HIGH-LEVEL FLOW



3. PRE-FLIGHT CHECKS

- 1 Check system health and dependencies.
- 2 Verify required tools, packages and permissions.
- 3 Validate input parameters.
- 4 Ensure backups or snapshots are available for risky changes.
- 5 Confirm maintenance window or approval if required.

Example: Pre-flight checks

```
#!/bin/bash
set -euo pipefail

# Check if running as root
if [[ $EUID -ne 0 ]]; then
    echo "Run as root"
    exit 1
fi

# Check required package
if ! command -v nginx &>/dev/null; then
    echo "nginx is required"
    exit 1
fi
```

4. EVIDENCE COLLECTION

- 1 Capture current configuration and system state.
- 2 Check relevant logs, metrics and service status.
- 3 Record network, disk, CPU and memory information if relevant.
- 4 Identify recent changes.
- 5 Save evidence for troubleshooting and rollback if needed.

Example: Collect evidence

```
mkdir -p /var/log/change-$(date +%F_%H%M)
systemctl status nginx > evidence/service.txt
ss -tuln > evidence/ports.txt
df -h > evidence/disk.txt
journalctl -u nginx --since "1 hour ago" \
> evidence/logs.txt
```

5. CONTROLLED MUTATION (MAKE SAFE CHANGES)

- 1 Change only what is necessary.
- 2 Use idempotent commands.
- 3 Validate configuration before applying.
- 4 Make small, incremental changes.
- 5 Handle failures and stop on errors.

Example: Safe configuration change

```
cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak
nginx -t # Validate config
if [ $? -eq 0 ]; then
    systemctl reload nginx
else
    echo "Config validation failed. No changes applied."
fi
exit 1
```

6. POST-CHANGE VALIDATION

- 1 Confirm the service is running as expected.
- 2 Check key functionality (health endpoints, logs, metrics).
- 3 Verify there are no new errors.
- 4 Compare current state with expected state.
- 5 Monitor for a period of time after the change.

Example: Validate after change

```
systemctl status nginx --no-pager
curl -s -o /dev/null -w "%(http_code)\n" \
http://localhost/health
if [ $? -eq 0 ]; then
    echo "Service is healthy"
else
    echo "Health check failed"
fi
exit 1
```

7. ROLLBACK TRIGGERS

- 1 Validation checks fail.
- 2 Service is unhealthy.
- 3 Error rate or latency increases.
- 4 Unexpected system behavior.
- 5 Any impact to users or critical functionality.

Example: Rollback

```
cp /etc/nginx/nginx.conf.bak \
/etc/nginx/nginx.conf
systemctl reload nginx
echo "Rolled back to previous config"
```



SENIOR ENGINEER TIP

Always design automation so that it can be safely stopped and rolled back. Collect enough evidence before and after the change to investigate any issues quickly.



REAL WORLD LESSON

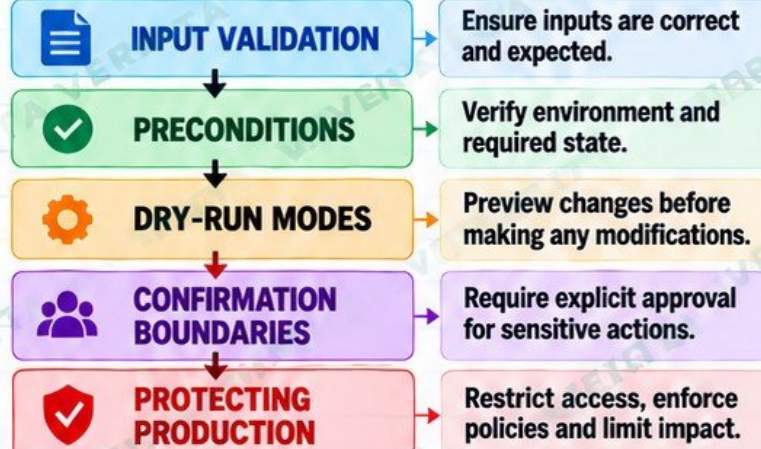
Many production incidents happen because changes were made without proper validation or rollback. A safe automation workflow reduces blast radius and makes it easier to recover.

04 Building Guardrails Into Automation

1. WHY GUARDRAILS MATTER

- ✓ Prevent accidental changes and human errors.
- ✓ Reduce blast radius when something goes wrong.
- ✓ Make automation safe, reliable and repeatable.
- ✓ Protect production systems and critical data.

2. GUARDRAIL LAYERS



3. INPUT VALIDATION

- 1 Check required parameters.
- 2 Validate format, type and allowed values.
- 3 Reject unexpected or dangerous input.
- 4 Sanitize inputs to prevent command injection.

Example: Validate input

```

if [[ -z "$1" ]]; then
  echo "Usage: $0 <server-name>"
  exit 1
fi
if [[ ! "$1" =~ ^[a-zA-Z0-9-]+$ ]]; then
  echo "Invalid server name: $1"
  exit 1
fi
  
```

4. PRECONDITIONS

- 1 Check required tools are installed.
- 2 Verify you are running as the correct user.
- 3 Confirm required services are available.
- 4 Validate system state before proceeding.

Example: Check prerequisites

```

command -v nginx >/dev/null 2>&1
|| { echo "nginx is required"; exit 1; }

if [[ $EUID -ne 0 ]]; then
  echo "Run as root"
  exit 1
fi
  
```

5. DRY-RUN MODES

- 1 Show what will change without making changes.
- 2 Useful for testing and review.
- 3 Helps detect unexpected impacts.
- 4 Default to dry-run for high-risk operations.

Example: Implement dry-run

```

DRY_RUN=false
if [[ "$1" == "--dry-run" ]]; then
  DRY_RUN=true
fi
if $DRY_RUN; then
  echo "[DRY-RUN] Would restart nginx service"
else
  systemctl restart nginx
fi
  
```

6. CONFIRMATION BOUNDARIES

- 1 Require explicit confirmation for risky actions.
- 2 Use clear and meaningful prompts.
- 3 Limit who can approve and execute changes.
- 4 Add timeouts to prevent accidental execution.

Example: Confirmation prompt

```

read -p "This will restart nginx. Continue? (yes/no): " ans
if [[ "$ans" != "yes" ]]; then
  echo "Operation cancelled"
  exit 1
fi
  
```

7. PROTECTING PRODUCTION

- 1 Use environment checks (dev, staging, prod).
- 2 Restrict execution to approved users or groups.
- 3 Implement allowlists and deny high-risk actions.
- 4 Log all actions and maintain an audit trail.

Example: Block execution on production

```

ENV=$(hostname -f)
if [[ "$ENV" == *"prod"* ]]; then
  echo "This script cannot be run on production"
  exit 1
fi
  
```



SENIOR ENGINEER TIP

Always fail safely. Validate everything, default to dry-run, and require confirmation for destructive actions. It is better to stop a change than to recover from a production incident.



REAL WORLD LESSON

Many production outages have been caused by scripts running with the wrong input, missing checks or without dry-run. Guardrails reduce human error and prevent small mistakes from becoming major incidents.

05

Exit Codes, Signals and Failure Propagation

1. EXIT CODES (\$?)

- ✓ Every command returns an exit code.
- ✓ 0 means success.
- ✓ Non-zero means failure.
- ✓ Use `$?` to check the exit code of the last command.

Example:

```
ls /etc/nginx
echo "Exit code: $?"

ls /nonexistent
echo "Exit code: $?"
```

```
Exit code: 0
ls: cannot access
/nonexistent:
No such file or
directory
Exit code: 2
```

2. set -e (EXIT ON ERROR)

- ✓ Stops the script immediately when a command fails (non-zero exit code).
- ✓ Prevent silent failures.
- ✓ Useful for production scripts.

Example:

```
#!/bin/bash
set -e
echo "Starting..."
ls /etc/nginx
echo "This will not run if a command above fails"
```

3. set -u (UNDEFINED VARIABLES)

- ✓ Stops the script if you use an undefined variable.
- ✓ Helps catch typos and missing inputs.
- ✓ Makes scripts more reliable.

Example:

```
#!/bin/bash
set -u
echo "User: $USER"
echo "Value: $UNDEFINED_VAR" # This will fail
```

4. set -o pipefail (PIPELINE FAILURES)

- ✓ By default, a pipeline returns the exit code of the last command only.
- ✓ `set -o pipefail` makes the pipeline fail if any command in the pipeline fails.
- ✓ Prevents hidden failures in pipelines.

Example:

```
#!/bin/bash
set -o pipefail
# Without pipefail, this may return 0 even if grep fails
cat /var/log/nginx/access.log | grep "ERROR" | wc -l
```

5. COMMON EXIT CODES

Code	Meaning
0	Success
1	General error
2	Misuse of shell command
126	Command found but not executable
127	Command not found
130	Terminated by Ctrl+C (SIGINT)
143	Terminated by SIGTERM (128 + 15)

6. SIGNALS: SIGTERM and SIGINT

- ✓ **SIGINT (2)** Sent when you press Ctrl+C.
 - Used for graceful shutdown.
- ✓ **SIGTERM (15)** Sent by default to stop a process.
 - Allows cleanup before exiting.
- ✓ Always handle signals in long-running scripts.

Example: Handle signals

```
#!/bin/bash
cleanup() {
    echo "Caught signal, cleaning up..."
    # add cleanup commands here
    exit 1
}
trap cleanup SIGINT SIGTERM
echo "Script is running..."
while true; do sleep 5; done
```

Common Signals

Signal	Number
SIGINT	2
SIGTERM	15
SIGKILL	9
SIGHUP	1

7. WHY SCRIPTS SILENTLY SUCCEED

- ✗ Not using `set -e` so the script continues after a failure.
- ✗ Pipelines hide failures (no `pipefail`).
- ✗ Errors are redirected (e.g. `2>/dev/null`).
- ✗ Commands in conditionals (if, while) do not trigger `set -e`.
- ✗ Background commands (&) can fail without stopping the script.

Example: Silent failure without pipefail

```
cat /nonexistent | grep error
echo "Exit code: $?" # Returns 0 (success) without pipefail
```

8. BEST PRACTICES

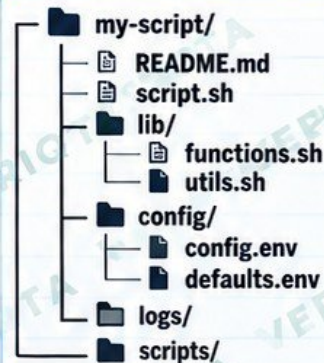
- ✓ Use `set -euo pipefail` for production scripts.
- ✓ Always check exit codes for critical commands.
- ✓ Handle signals and perform cleanup.
- ✓ Validate inputs before running commands.
- ✓ Log errors with context.
- ✓ Use functions and traps for safe shutdown.
- ✓ Test scripts in non-production first.

Designing Maintainable Bash Automation

1. WHY DESIGN FOR MAINTAINABILITY?

- ✓ Easier to read, understand and modify.
- ✓ Reduces errors and operational risk.
- ✓ Makes scripts reusable and testable.
- ✓ Helps teams collaborate on automation.
- ✓ Supports long-term reliability in production.

2. SCRIPT STRUCTURE (RECOMMENDED)



Key sections in a script:

- 1 Shebang and description
- 2 Default variables and config
- 3 Functions
- 4 Main logic
- 5 Argument parsing
- 6 Logging
- 7 Error handling
- 8 Cleanup and exit

3. FUNCTIONS

- ✓ Break your script into reusable functions.
- ✓ Make code easier to read and maintain.
- ✓ Use clear, descriptive function names.
- ✓ Return meaningful exit codes.

Example: Function

```

#!/bin/bash

check_package() {
    local package=$1
    if dpkg -s "$package" >/dev/null; then
        echo "Package $package is installed"
        return 0
    else
        echo "Package $package is not installed"
        return 1
    fi
}

# Usage
check_package "nginx"
  
```

4. readonly (CONSTANT VARIABLES)

- ✓ Prevent accidental changes to important variables.
- ✓ Useful for configuration values and paths.
- ✓ Makes scripts safer and more predictable.

Example: readonly

```

#!/bin/bash

readonly APP_NAME="webapp"
readonly CONFIG_FILE="/etc/webapp.conf"
readonly LOG_DIR="/var/log/webapp"

echo "App: $APP_NAME"
echo "Config: $CONFIG_FILE"
echo "Log Dir: $LOG_DIR"
  
```

5. local (FUNCTION SCOPE)

- ✓ Declare variables as local inside functions.
- ✓ Prevent variable conflicts.
- ✓ Keeps your script predictable.

Example: local variable

```

#!/bin/bash

process_user() {
    local user="$1"
    local home_dir
    home_dir=$(getent passwd "$user" | cut -d: -f6)
    echo "User: $user"
    echo "Home: $home_dir"
}

# Usage
process_user "nginx"
  
```

6. PREDICTABLE INTERFACES

- ✓ Use command-line arguments with clear options.
- ✓ Validate inputs.
- ✓ Provide helpful usage messages.
- ✓ Return proper exit codes.

Example: Argument parsing

```

#!/bin/bash

usage() {
    echo "Usage: $0 -u <user> -d <dir>"
    exit 1
}

while getopts ":u:d:h" opt; do
    case $opt in
        u) USER="$OPTARG" ;;
        d) DIR="$OPTARG" ;;
        h) usage ;;
        *) usage ;;
    esac
done

echo "User: $USER"
echo "Directory: $DIR"
  
```

7. COMPLETE EXAMPLE: WELL-STRUCTURED SCRIPT

```

#!/bin/bash
# Description: Install and verify a package
# Author: VERIQTA
set -euo pipefail

readonly PACKAGE="nginx"
readonly LOG_FILE="/var/log/install.log"

log() {
    local msg="$1"
    echo "$(date +%F %T) - $msg" | tee -a "$LOG_FILE"
}

install_package() {
    if dpkg -s "$PACKAGE" >/dev/null; then
        log "Package $PACKAGE already installed"
        return 0
    fi
    log "Installing $PACKAGE..."
    sudo apt update
    sudo apt install -y "$PACKAGE"
    log "Installation completed"
}

main() {
    log "Starting script"
    install_package
    log "Script finished successfully"
}

main "$@"
  
```

What this example shows:

- 1 Shebang and description
- 2 set -euo pipefail
- 3 readonly variables
- 4 Logging function
- 5 Reusable function
- 6 Proper error handling
- 7 Clear main function
- 8 Predictable structure



SENIOR ENGINEER TIP

Write your scripts as if another engineer will maintain them. Use functions, clear names, input validation and proper error handling.



REAL WORLD LESSON

Poorly structured scripts with global variables, no input validation and unclear logic are hard to maintain, easy to break and often cause production incidents.

1. WHY DEFENSIVE PATTERNS?

- ✓ Prevent unexpected behavior in production.
- ✓ Avoid data corruption and security issues.
- ✓ Make scripts more reliable, predictable and maintainable.
- ✓ Handle edge cases and untrusted input.
- ✓ Reduce failures when working with files, users and external commands.

2. QUOTING (HANDLE SPACES AND SPECIAL CHARACTERS)

- ✓ Always quote variables.
- ✓ Use double quotes for variables.
- ✓ Use single quotes for literals.
- ✓ Prevent word splitting and globbing.
- ✓ Keep commands predictable and safe.

Example:

```
name="John Doe"
echo "$name"      # Correct: keeps the space
echo $name        # Wrong: splits into two words
file="/path/with spaces/file.txt"
cat "$file"       # Correct
cat $file         # Wrong: tries to read /path/with
```

3. GLOBBING (WILDCARDS)

- ✓ Globbing expands patterns like * ? [].
- ✓ Can match multiple files unexpectedly.
- ✓ May cause commands to act on wrong files.
- ✓ Use quotes or disable globbing when needed.

Example:

```
# This may delete more files than expected
rm *.log

# Safe: prevents globbing
rm "*.log"

# Disable globbing temporarily
set -f
rm *.log      # Treated as a literal string
set +f
```

4. WORD SPLITTING (IFS)

- ✓ Unquoted variables are split into words based on IFS (space, tab, newline).
- ✓ Can break your script with unexpected input.
- ✓ Always quote variables.
- ✓ Be careful when reading input or command output.

Example:

```
files="file1.txt file2.txt"
# Wrong: splits into two words
for f in $files; do
    echo "$f"
done
# Correct: keeps the value as one string
for f in "$files"; do
    echo "$f"
done
```

5. TEMPORARY FILES (SAFE USAGE)

- ✓ Use secure methods to create temporary files.
- ✓ Avoid hardcoded file names.
- ✓ Ensure proper cleanup with traps.
- ✓ Prevent sensitive data from being exposed.

Example:

```
# Create a temporary file safely
tmpfile=$(mktemp)
echo "Processing data..." > "$tmpfile"
# Use the file
cat "$tmpfile"
# Clean up
rm -f "$tmpfile"
```

6. RACE CONDITIONS (CONCURRENT ACCESS)

- ✓ Happen when multiple processes access the same resource at the same time.
- ✓ Can cause data corruption or unexpected behavior.
- ✓ Use locks or atomic operations.
- ✓ Be careful with temporary files and shared resources.

Example: Using a lock file

```
lockfile="/tmp/my_script.lock"
if ( set -o noclobber; echo "$$" > "$lockfile" ) 2>/dev/null; then
    trap 'rm -f "$lockfile"' EXIT
    echo "Running script..."
    # Your script logic here
else
    echo "Another instance is already running."
    exit 1
fi
```



SENIOR ENGINEER TIP

Always assume your script will handle unexpected input, special characters, multiple users and concurrent executions. Use defensive patterns by default.



REAL WORLD LESSON

Many production incidents happen because scripts did not handle spaces, special characters, concurrent runs or temporary files correctly. Defensive patterns prevent these common but serious issues.

Traps and Reliable Cleanup

1. WHY USE TRAPS?

- ✓ Ensure cleanup happens even when the script exits unexpectedly.
- ✓ Handle errors and signals gracefully.
- ✓ Prevent leftover files, processes and partial state.
- ✓ Make scripts more reliable in production.
- ✓ Useful for temporary files, background processes and service changes.

2. TRAP SYNTAX

```
trap 'command' SIGNALS
```

Common signals:

- 1 EXIT (script exit)
- 2 ERR (command error)
- 3 INT (Ctrl + C)
- 4 TERM (termination signal)

Syntax examples:

```
trap 'cleanup' EXIT
trap 'echo "Error occurred"' ERR
trap 'echo "Interrupted"; cleanup' INT
trap 'echo "Terminated"; cleanup' TERM
```

3. CLEANUP FUNCTION

- 1 Put all cleanup logic in a function.
- 2 Remove temporary files and directories.
- 3 Stop background processes.
- 4 Revert partial changes.
- 5 Always make cleanup idempotent (it should be safe to run multiple times).

Example: Cleanup function

```
#!/bin/bash
cleanup() {
    echo "Running cleanup..."
    rm -f "$TMP_FILE" 2>/dev/null
    kill "$BACKGROUND_PID" 2>/dev/null
    echo "Cleanup completed."
}
```

4. COMPLETE EXAMPLE

```
#!/bin/bash
set -euo pipefail

# Variables
TMP_FILE=$(mktemp)
BACKGROUND_PID=""

cleanup() {
    echo "Cleaning up..."
    [[ -n "$TMP_FILE" ]] && rm -f "$TMP_FILE"
    [[ -n "$BACKGROUND_PID" ]] && kill "$BACKGROUND_PID" 2>/dev/null
    echo "Cleanup completed."
}

# Set traps
trap cleanup EXIT
trap 'echo "Error on line $LINENO"' ERR
trap 'echo "Interrupted by user"; exit 1' INT
trap 'echo "Terminated"; exit 1' TERM

# Example usage
echo "Starting work..."
sleep 100 &
BACKGROUND_PID=$!
echo "Background process started (PID: $BACKGROUND_PID)"
echo "Doing some work..."
sleep 10
echo "Script completed successfully."
```

5. HANDLING PARTIAL STATE

- ✓ Track what has been created or changed.
- ✓ Clean up only if it exists.
- ✓ Use checks to avoid deleting important data.
- ✓ Make cleanup safe to run multiple times.

Example: Clean partial state

```
cleanup() {
    [[ -d "/tmp/mydir" ]] && rm -rf "/tmp/mydir"
    [[ -f "/tmp/myfile" ]] && rm -f "/tmp/myfile"
    [[ -n "$PID" ]] && kill "$PID" 2>/dev/null
}
```

6. TESTING TRAPS

- ✓ Test normal exit (script finishes).
- ✓ Test error exit (command fails).
- ✓ Test Ctrl + C (INT).
- ✓ Test termination (TERM).
- ✓ Verify all temporary files and processes are cleaned up.

Test commands:

```
# Test normal exit
./script.sh
# Test error
bash -c 'set -e; false'
# Test Ctrl + C
./script.sh (press Ctrl + C)
# Test termination
kill -TERM <script_pid>
```



SENIOR ENGINEER TIP

Always clean up everything you create, even if the script fails or is interrupted. Use traps to handle unexpected exits and signals. Make cleanup idempotent and safe to run multiple times.



REAL WORLD LESSON

Many production incidents happen because temporary files, background processes or partial changes are left behind when scripts fail or are terminated. Reliable cleanup prevents resource leaks and ensures systems stay in a consistent state.

Structured Logging for Automation

1. WHY STRUCTURED LOGGING?

- ✓ Makes automation easier to debug and troubleshoot.
- ✓ Provides clear evidence during incidents.
- ✓ Helps track what happened, when and why.
- ✓ Supports monitoring, alerting and auditing.
- ✓ Useful for both humans and machines (to parse and query).

2. TIMESTAMPS

- ✓ Always include a timestamp in every log.
- ✓ Use a consistent format (ISO 8601).
- ✓ Helps correlate events across systems.
- ✓ Use date command in Bash.

Example: Timestamp in logs

```
echo "$(date -u +%Y-%m-%dT%H:%M:%SZ)" [INFO] \
"Starting backup..."
```

Output:

```
2026-09-28T14:30:05Z [INFO] Starting backup...
```

3. SEVERITY LEVELS

- ✓ Use clear severity levels.
- ✓ Common levels: DEBUG, INFO, WARN, ERROR, FATAL.
- ✓ Helps filter and search logs.
- ✓ Be consistent across all scripts.

Example: Logging with severity

```
log() {
  level="$1"
  message="$2"
  echo "$(date -u +%Y-%m-%dT%H:%M:%SZ)"
  [$level] $message"
}
```

```
log "INFO" "Backup started"
log "WARN" "Low disk space"
log "ERROR" "Failed to copy file"
```

4. STDOUT VERSUS STDERR

- ✓ Use stdout for normal logs (INFO, DEBUG).
- ✓ Use stderr for errors and warnings.
- ✓ Helps separate normal output from error messages.
- ✓ Useful when redirecting and storing logs.

Example: stdout vs stderr

```
echo "This is an info message"
echo "This is a warning" >&2
echo "This is an error" >&2
```

Redirect output to files:

```
./script.sh > output.log >> error.log
```

5. CORRELATION IDS

- ✓ Use a unique ID to track related log entries.
- ✓ Helps trace a single execution across multiple steps.
- ✓ Useful in distributed systems and large automation.
- ✓ Can be a timestamp, UUID or a provided ID.

Example: Using a correlation ID

```
CORR_ID=$(uuidgen)
log() {
  level="$1"
  message="$2"
  echo "$(date -u +%Y-%m-%dT%H:%M:%SZ)"
  [$level] [correlation_id=$CORR_ID] $message"
}
log "INFO" "Starting deployment"
```

6. SYSTEMD JOURNALD INTEGRATION

- ✓ Use logger command to send logs to journald.
- ✓ Logs can be viewed with journalctl.
- ✓ Useful for production services and scripts running as systemd units.
- ✓ Supports structured fields (like PRIORITY, SYSLOG_IDENTIFIER).

Example: Log to journald

```
logger -t "my-script" -p info "Backup completed successfully"
logger -t "my-script" -p err "Failed to copy file"
# View logs
journalctl -t my-script -n 20
```

7. COMPLETE EXAMPLE: STRUCTURED LOGGING SCRIPT

```
#!/bin/bash
SCRIPT_NAME="backup"
CORR_ID=$(uuidgen)
log() {
  level="$1"
  message="$2"
  echo "$(date -u +%Y-%m-%dT%H:%M:%SZ)"
  [$level] [correlation_id=$CORR_ID] [script=$SCRIPT_NAME] \
  $message"
}
log "INFO" "Starting backup process"
log "INFO" "Source: /data"
if tar -czf /backup/data.tar.gz /data 2>>/var/log/backup.err; then
  log "INFO" "Backup completed successfully"
else
  log "ERROR" "Backup failed"
fi exit 1
```



SENIOR ENGINEER TIP

Use structured, consistent logging in all automation. Include timestamps, severity, correlation IDs and send important logs to journald for better observability.



REAL WORLD LESSON

During incidents, clear and structured logs with timestamps and correlation IDs can reduce troubleshooting time from hours to minutes.

1. WHY USE LOCKING?

- ✓ Prevents duplicate executions of the same script.
- ✓ Avoids data corruption and race conditions.
- ✓ Useful for cron jobs and automated tasks.
- ✓ Ensures only one instance runs at a time.
- ✓ Helps maintain system stability and predictable behavior.

2. USING FLOCK (RECOMMENDED)

- ✓ **flock** is a simple and reliable way to prevent concurrent executions.
- ✓ Works with a lock file and a command or script.
- ✓ Automatically releases the lock when the script exits.
- ✓ Useful for cron jobs and long-running scripts.

Example: Using flock

```
#!/bin/bash
exec 200>/var/lock/my_script.lock
flock -n 200 || { echo "Another instance is running.";
  exit 1; }

echo "Script started at $(date)"
# Your script logic here
echo "Script completed at $(date)"
```

3. LOCK FILES (MANUAL METHOD)

- ✓ Create a lock file to indicate the script is running.
- ✓ Check if the lock file exists before starting.
- ✓ Remove the lock file when the script exits.
- ✓ Useful when flock is not available.

Example: Using a lock file

```
#!/bin/bash
LOCK_FILE="/var/lock/my_script.lock"

if [ -e "$LOCK_FILE" ]; then
  echo "Another instance is running."
  exit 1
fi

# Create lock file
echo $$ > "$LOCK_FILE"

# Your script logic here

# Remove lock file
rm -f "$LOCK_FILE"
```

4. HANDLING STALE LOCKS

- ✓ Stale locks can remain if a script crashes or is killed.
- ✓ Check if the process is still running.
- ✓ Remove stale locks safely.
- ✓ Use process IDs (PIDs) to verify.

Example: Handling stale locks

```
#!/bin/bash
LOCK_FILE="/var/lock/my_script.lock"

if [ -e "$LOCK_FILE" ]; then
  OLD_PID=$(cat "$LOCK_FILE")
  if ps -p "$OLD_PID" > /dev/null 2>&1; then
    echo "Another instance (PID: $OLD_PID) is running."
    exit 1
  else
    echo "Stale lock found. Removing..."
    rm -f "$LOCK_FILE"
  fi
fi

echo $$ > "$LOCK_FILE"
# Your script logic here
```

5. OVERLAPPING CRON JOBS

- ✓ Cron can start a new instance even if the previous one is still running.
- ✓ Use flock or lock files to prevent overlaps.
- ✓ Set reasonable timeouts if needed.
- ✓ Monitor logs to detect skipped runs.

Example: Cron job with flock

```
# Run script every 5 minutes with flock
*/5 * * * * /usr/bin/flock -n /var/lock/my_script.lock \
/path/to/my_script.sh >> /var/log/my_script.log 2>&1
```

6. COMPLETE EXAMPLE: SAFE SCRIPT

```
#!/bin/bash
LOCK_FILE="/var/lock/backup.lock"

exec 200>"$LOCK_FILE"
flock -n 200 || { echo "Backup already running."; exit 1; }

trap 'rm -f "$LOCK_FILE"' EXIT

echo "Backup started at $(date)"
# Backup logic here
sleep 30
echo "Backup completed at $(date)"
```



SENIOR ENGINEER TIP

Always use flock for simplicity and reliability. Handle stale locks, use traps for cleanup and monitor logs to ensure your automation does not run twice.



REAL WORLD LESSON

Overlapping cron jobs can cause data corruption, unexpected behavior and high resource usage. Use locking to ensure only one instance runs at a time.

1. WHY SAFE FILE AUTOMATION?

- ✓ Prevents data loss and corruption.
- ✓ Ensures changes are applied reliably and safely.
- ✓ Maintains correct ownership and permissions.
- ✓ Allows easy recovery if something goes wrong.
- ✓ Important for production systems and automated scripts.

2. ATOMIC WRITES (SAFE FILE REPLACEMENT)

- ✓ Write to a temporary file first.
- ✓ Validate the content (optional).
- ✓ Move the temporary file to the final location.
- ✓ Ensures the file is either fully written or not changed at all.

Example: Atomic write

```
#!/bin/bash
FILE="/etc/myapp/config.conf"
TMP=$(mktemp)
echo "new configuration content" > "$TMP"
# Optional: validate content here
mv "$TMP" "$FILE"
echo "File updated atomically."
```

3. TEMPORARY FILES (SAFE USAGE)

- ✓ Use **mktemp** to create unique temporary files.
- ✓ Avoid predictable file names.
- ✓ Ensure proper cleanup even if the script fails.
- ✓ Store temporary files in a safe directory (e.g. /tmp).

Example: Using temporary files

```
#!/bin/bash
TMP_FILE=$(mktemp /tmp/myapp.XXXXXX)
echo "Processing data..." > "$TMP_FILE"
# Do some work with $TMP_FILE
# Always clean up
rm -f "$TMP_FILE"
```

4. OWNERSHIP (FILE OWNERSHIP)

- ✓ Set the correct owner for files and directories.
- ✓ Use **chown** to change ownership.
- ✓ Important for system files, application files and sensitive data.

Example: Setting file ownership

```
#!/bin/bash
FILE="/etc/myapp/config.conf"
chown appuser:appgroup "$FILE"
echo "Ownership set to appuser:appgroup"
```

5. PERMISSIONS (FILE PERMISSIONS)

- ✓ Use **chmod** to set the correct permissions.
- ✓ Follow the principle of least privilege.
- ✓ Avoid world-writable files.
- ✓ Set appropriate permissions for scripts and configs.

Example: Setting file permissions

```
#!/bin/bash
FILE="/etc/myapp/config.conf"
chmod 640 "$FILE"
echo "Permissions set to 640"
```

6. BACKUPS BEFORE MUTATION

- ✓ Always create a backup before modifying a critical file.
- ✓ Use a timestamp or versioning in the backup name.
- ✓ Store backups in a safe location.
- ✓ Helps with quick recovery if something goes wrong.

Example: Creating a backup

```
#!/bin/bash
FILE="/etc/myapp/config.conf"
BACKUP_DIR="/var/backups/myapp"
mkdir -p "$BACKUP_DIR"
BACKUP_FILE="$BACKUP_DIR/config.conf.$(date +%Y%m%d%H%M%S)"
cp "$FILE" "$BACKUP_FILE"
echo "Backup created: $BACKUP_FILE"
```

7. COMPLETE EXAMPLE: SAFE FILE UPDATE

```
#!/bin/bash
set -euo pipefail
FILE="/etc/myapp/config.conf"
BACKUP_DIR="/var/backups/myapp"
mkdir -p "$BACKUP_DIR"
# 1. Create backup
cp "$FILE" "$BACKUP_DIR/config.conf.$(date +%Y%m%d%H%M%S)"
# 2. Write to temporary file
TMP=$(mktemp)
echo "new configuration content" > "$TMP"
# 3. Set ownership and permissions
chown appuser:appgroup "$TMP"
chmod 640 "$TMP"
# 4. Move to final location (atomic write)
mv "$TMP" "$FILE"
echo "File updated successfully."
```



SENIOR ENGINEER TIP

Always use atomic writes, temporary files, correct ownership and permissions, and create backups before making changes. This reduces the risk of data loss and makes your automation scripts reliable and safe to run in production.



REAL WORLD LESSON

Accidental file overwrites, wrong permissions or missing backups can cause application failures, security issues and downtime. Always follow safe file automation practices.

Production Text Processing

1. WHY TEXT PROCESSING MATTERS?

- ✓ Logs, configs and system output are mostly text.
- ✓ Helps you extract useful information quickly.
- ✓ Essential for troubleshooting and automation.
- ✓ Works on large files and in pipelines.
- ✓ Reduces manual work and saves time in production.

2. GREP (SEARCH TEXT)

- ✓ Search for a pattern in files or input.
- ✓ Supports regular expressions.
- ✓ Useful for logs and large files.
- ✓ Shows matching lines only.

Example: Using grep

```
# Search for 'error' in a log file
grep -i "error" /var/log/app.log

# Show line numbers
grep -n "error" /var/log/app.log

# Search recursively in a directory
grep -rn "error" /var/log/
```

3. SED (STREAM EDITOR)

- ✓ Find and replace text.
- ✓ Edit text in a stream (no need to open the file).
- ✓ Useful for simple text transformations.
- ✓ Works well in scripts and pipelines.

Example: Using sed

```
# Replace 'old' with 'new'
sed 's/old/new/g' file.txt

# Delete empty lines
sed '/^$/d' file.txt

# Print specific lines (e.g. 5 to 10)
sed -n '5,10p' file.txt
```

4. AWK (PATTERN SCANNING)

- ✓ Process structured text like columns.
- ✓ Powerful for filtering, calculations and formatting.
- ✓ Great for logs, reports and CSV-like data.

Example: Using awk

```
# Print first column
awk '{print $1}' file.txt

# Print first and third column
awk '{print $1, $3}' file.txt

# Find lines where column 3 > 100
awk '$3 > 100' file.txt
```

5. CUT (EXTRACT COLUMNS)

- ✓ Extract specific columns from text.
- ✓ Useful for simple, structured data (e.g. CSV, /etc/passwd).
- ✓ Fast and easy to use.

Example: Using cut

```
# Extract first column (space delimited)
cut -d ' ' -f 1 file.txt

# Extract specific columns (comma separated)
cut -d ',' -f 1,3 file.csv

# Extract username from /etc/passwd
cut -d ':' -f 1 /etc/passwd
```

6. SORT (SORT LINES)

- ✓ Sort lines in alphabetical or numeric order.
- ✓ Useful for organizing output.
- ✓ Works with other tools in pipelines.

Example: Using sort

```
# Sort lines (alphabetical)
sort file.txt

# Sort numerically
sort -n numbers.txt

# Sort in reverse order
sort -r file.txt

# Sort by a specific column (column 2)
sort -k2 file.txt
```

7. UNIQ (REMOVE DUPLICATES)

- ✓ Remove duplicate lines.
- ✓ Often used with sort.
- ✓ Useful for finding unique entries in logs or reports.

Example: Using uniq

```
# Remove duplicate lines
# (note: works best with sorted input)
sort file.txt | uniq

# Show only duplicate lines
sort file.txt | uniq -d

# Show only unique lines
sort file.txt | uniq -u

# Show count of each line
sort file.txt | uniq -c
```

8. CHOOSING THE CORRECT TOOL

- ✓ Use **grep** to search for patterns.
- ✓ Use **sed** for simple text replacement.
- ✓ Use **awk** for complex processing.
- ✓ Use **cut** for extracting columns.
- ✓ Use **sort** to organize output.
- ✓ Use **uniq** to remove or find duplicates.

Quick Guide

Task	Best Tool
Search for text	grep
Replace text	sed
Process columns / calculations	awk
Extract specific columns	cut
Sort data	sort
Remove duplicates	uniq



SENIOR ENGINEER TIP

Combine these tools in pipelines to handle real-world data. For example: **grep | awk | sort | uniq** can quickly filter, process and summarize large log files.



REAL WORLD LESSON

In production, logs can be huge. Using the right tool (grep, sed, awk, cut, sort, uniq) helps you quickly find issues, reduce manual work and automate troubleshooting.

13 Automating Structured Data

JSON with jq | YAML with yq | Avoiding Fragile Regex Parsing

1. WHY USE STRUCTURED DATA TOOLS?

- ✓ Reliable and consistent parsing of JSON and YAML.
- ✓ Handles complex data easily.
- ✓ Reduces errors compared to regex parsing.
- ✓ Essential for modern DevOps and automation workflows.
- ✓ Works well with APIs, configs and infrastructure files.

2. WORKING WITH JSON USING jq

- ✓ jq is a powerful tool for querying and transforming JSON.
- ✓ Supports filtering, extraction, modification and formatting.
- ✓ Ideal for API responses, logs and config files.
- ✓ Produces clean and predictable output.

Example: Parsing JSON with jq

```
# Sample JSON file
cat data.json
{ "name": "server1", "ip": "10.0.0.1", "tags": ["web", "prod"] }

# Extract a value
jq -r '.name' data.json

# Extract multiple fields
jq -r '.name + " - " + .ip' data.json

# Get all tags
jq -r '.tags[]' data.json
```

3. COMMON jq USE CASES

- ✓ Extract specific fields.
- ✓ Filter arrays and objects.
- ✓ Modify or add values.
- ✓ Format output for scripts.

Example: More jq examples

```
# Get all server names from an array
jq -r '.servers[].name' servers.json

# Filter by condition
jq -r '.servers[] | select(.env=="prod") | .name' \
servers.json

# Pretty print JSON
jq . data.json
```

4. WORKING WITH YAML USING yq

- ✓ yq is the YAML equivalent of jq.
- ✓ Used for reading, modifying and querying YAML files.
- ✓ Very useful for Kubernetes, Docker Compose and CI/CD configs.
- ✓ Works with both YAML v1 and v2 formats.

Example: Parsing YAML with yq

```
# Sample YAML file
cat config.yaml
app:
  name: myapp
  replicas: 3
  image: nginx:1.25
  env:
    - name: ENV
      value: prod

# Get a value
yq '.app.name' config.yaml

# Update a value
yq '.app.replicas = 5' config.yaml

# Get multiple fields
yq '.app | {name, replicas, image}' \
config.yaml

# Get environment variables
yq '.app.env[]' config.yaml
```

5. COMMON yq USE CASES

- ✓ Read and extract values.
- ✓ Update existing values.
- ✓ Add new fields.
- ✓ Work with complex YAML structures.

Example: More yq examples

```
# Extract image
yq '.app.image' config.yaml

# Update image
yq '.app.image = "nginx:1.26"' config.yaml

# Add a new field
yq '.app.version = "1.0.0"' config.yaml
```

6. AVOIDING FRAGILE REGEX PARSING

- ✓ Regex can break with nested structures or different formats.
- ✓ JSON and YAML have proper parsers (jq and yq).
- ✓ Regex may work for simple cases but is not reliable for production.
- ✓ Use jq and yq for predictable and maintainable scripts.

Bad vs Good Example

✗ Fragile regex approach (bad)

```
# Trying to extract a JSON value
# with regex (not reliable)
cat data.json | grep -o '"name": "[^"]*" \|
sed "s/\"name\": \"\"/"
```

This can break with spaces, nested fields or different formats.

✓ Use jq instead (good)

```
jq -r '.name' data.json
```

- Reliable
- Handles nested structures
- Works with any valid JSON
- Easier to maintain

7. WHEN TO USE WHAT?

Data Type	Tool	Common Use Cases
JSON	jq	API responses, logs, cloud output (AWS, Azure, GCP), application data
YAML	yq	Kubernetes manifests, Docker Compose, CI/CD configs, infrastructure as code
Text (simple)	grep/sed/awk	Plain text logs, quick checks
Avoid	regex for JSON/YAML	Complex or nested data (use jq or yq)

8. PRACTICAL EXAMPLE: END-TO-END SCRIPT

```
#!/bin/bash
# Extract image from YAML and update JSON config
IMAGE=$(yq -r '.app.image' config.yaml)
echo "Image from YAML: $IMAGE"

# Update JSON with new image
jq --arg img "$IMAGE" '.image = $img' deployment.json \
> deployment-updated.json
echo "Updated JSON saved to deployment-updated.json"
```

SENIOR ENGINEER TIP
Always use jq for JSON and yq for YAML. Avoid regex parsing for structured data. It reduces errors, makes your scripts more reliable and easier to maintain.

REAL WORLD LESSON
Production incidents have been caused by fragile regex parsing that broke when the data format changed. Use proper tools like jq and yq to build robust and future-proof automation.

14 Configuration File Automation

Validate before replacing | Diff before change | Backup | Atomic replacement | Service validation

1. WHY SAFE CONFIGURATION AUTOMATION?

- ✓ Prevents broken configurations.
- ✓ Reduces downtime and failed deployments.
- ✓ Provides a clear change history.
- ✓ Makes automation safe and repeatable.
- ✓ Essential for production systems and critical services.

2. VALIDATE BEFORE REPLACING

- ✓ Always validate the new configuration before applying.
- ✓ Use built-in validation commands for each service.
- ✓ Catches syntax errors and prevents service failures.
- ✓ Faster rollback and recovery.

Example: Validate Nginx configuration

```
# Test Nginx configuration without applying
sudo nginx -t
# Output
# nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
# nginx: configuration file /etc/nginx/nginx.conf test is successful
```

3. DIFF BEFORE CHANGE

- ✓ Compare the current and new configuration.
- ✓ Shows exactly what will change.
- ✓ Helps you review and catch unexpected changes.
- ✓ Useful for auditing and change approval.

Example: Using diff

```
# Compare current config with new config
diff -u /etc/nginx/nginx.conf /tmp/nginx.conf.new
# Side-by-side view (easier to read)
diff -y /etc/nginx/nginx.conf /tmp/nginx.conf.new
```

4. CREATE A BACKUP

- ✓ Always create a backup before making changes.
- ✓ Use timestamps for easy recovery.
- ✓ Store backups in a safe location.
- ✓ Helps you quickly restore if something goes wrong.

Example: Create a backup

```
# Create a timestamped backup
sudo cp /etc/nginx/nginx.conf
/etc/nginx/nginx.conf.bak.$(date +%Y%m%d-%H%M%S)
# List backups
ls -lh /etc/nginx/nginx.conf.bak.*
```

5. ATOMIC REPLACEMENT (SAFE UPDATE)

- ✓ Write changes to a temporary file first.
- ✓ Validate the temporary file.
- ✓ Move the file to the final location (atomic operation).
- ✓ Prevents partial writes and corrupted files.

Example: Atomic configuration update

```
# 1. Create temporary file
TMP_FILE="/tmp/nginx.conf.new"
# 2. Write new configuration (example)
cat > $TMP_FILE <<'EOF'
# new nginx configuration
worker_processes auto;
EOF
# 3. Validate the configuration
sudo nginx -t -c $TMP_FILE
# 4. Move to final location (atomic replace)
sudo mv $TMP_FILE /etc/nginx/nginx.conf
```

6. SERVICE VALIDATION

- ✓ Test the configuration before restarting the service.
- ✓ Reload or restart only if validation passes.
- ✓ Check service status after the change.
- ✓ Ensures the service is running correctly.

Example: Validate and reload service

```
# Validate first
sudo nginx -t
# Reload if valid (no downtime)
sudo systemctl reload nginx
# Check service status
sudo systemctl status nginx --no-pager
```

7. COMPLETE EXAMPLE: CONFIGURATION UPDATE SCRIPT

```
#!/bin/bash
CONFIG_FILE="/etc/nginx/nginx.conf"
TMP_FILE="/tmp/nginx.conf.new"
BACKUP_FILE="${CONFIG_FILE}.bak.$(date +%Y%m%d-%H%M%S)"

# 1. Create backup
sudo cp "$CONFIG_FILE" "$BACKUP_FILE"
echo "Backup created: $BACKUP_FILE"

# 2. Generate new configuration (example)
cat > "$TMP_FILE" <<'EOF'
worker_processes auto;
EOF

# 3. Validate new configuration
sudo nginx -t -c "$TMP_FILE"
if [ $? -ne 0 ]; then
    echo "Validation failed. Keeping original config."
    rm -f "$TMP_FILE"
    exit 1
fi

# 4. Atomic replace
sudo mv "$TMP_FILE" "$CONFIG_FILE"
echo "Configuration updated successfully."

# 5. Reload and verify service
sudo systemctl reload nginx
sudo systemctl status nginx --no-pager
```

8. KEY PRINCIPLES

- ✓ Validate before replacing.
- ✓ Diff before change.
- ✓ Always create a backup.
- ✓ Use atomic replacement.
- ✓ Validate and reload the service.
- ✓ Handle errors and rollback if needed.
- ✓ Log all actions for auditability.



REAL WORLD LESSON

A simple syntax error in a configuration file can take down a production service. Always validate, diff, backup and use atomic replacement to keep your systems safe and available.



SENIOR ENGINEER TIP

Use validation, diff, backup, atomic replacement and service checks in all configuration automation. This prevents downtime, reduces risk and makes your scripts reliable in production.



VERIQTA



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

15 Detecting and Correcting Configuration Drift

Desired versus actual state | Checksums | Remediation | Audit trails | When not to auto-remediate

1. WHAT IS CONFIGURATION DRIFT?

- ✓ Configuration drift happens when the actual state of a system or file is different from the desired state.
- ✓ It can occur due to manual changes, failed deployments, package updates or unauthorized modifications.
- ✓ Drift can cause security issues, service failures and inconsistent behavior across environments.
- ✓ Detecting and correcting drift keeps your systems stable, secure and predictable.

2. DESIRED VERSUS ACTUAL STATE

- ✓ Desired state is the expected configuration (e.g. from Git, a template or a standard).
- ✓ Actual state is what is currently on the system.
- ✓ Compare both to detect differences (drift).
- ✓ Store desired state in a version-controlled repository.
- ✓ Use checksums, diff or configuration management tools to compare.

Example: Compare desired vs actual file

```
# Calculate checksum of desired and actual files
sha256sum /path/desired.conf
sha256sum /etc/myapp/myapp.conf

# Compare the files
diff -u /path/desired.conf /etc/myapp/myapp.conf
```

3. USING CHECKSUMS

- ✓ Checksums help detect if a file has changed.
- ✓ Common tools: md5sum, sha1sum, sha256sum.
- ✓ Store the expected checksum and compare with the current file.
- ✓ Useful for critical configuration files and scripts.

Example: Using checksums

```
# Generate checksum for a file
sha256sum /etc/myapp/myapp.conf

# Verify against a stored checksum
EXPECTED="a3f5...d9c1"
CURRENT=$(sha256sum /etc/myapp/myapp.conf | cut -d' ' -f1)

if [ "$EXPECTED" = "$CURRENT" ]; then
    echo "File is unchanged."
else
    echo "Configuration drift detected!"
fi
```

4. REMEDIATION (CORRECTING DRIFT)

- ✓ Remediation brings the actual state back to the desired state.
- ✓ Can be automatic or manual.
- ✓ Always back up before making changes.
- ✓ Validate configuration after remediation.
- ✓ Log all remediation actions for audit purposes.

Example: Remediation with backup and validation

```
#!/bin/bash
CONFIG_FILE="/etc/myapp/myapp.conf"
DESIRED_FILE="/path/desired.conf"
BACKUP_DIR="/var/backups/config"

# Create backup
mkdir -p "$BACKUP_DIR"
cp "$CONFIG_FILE" "$BACKUP_DIR/myapp.conf.$(date +%Y%m%d-%H%M%S)"

# Replace with desired configuration (atomic)
sudo cp "$DESIRED_FILE" "$CONFIG_FILE.tmp"
sudo mv "$CONFIG_FILE.tmp" "$CONFIG_FILE"

# Validate configuration (example for nginx)
sudo nginx -t && sudo systemctl reload nginx
echo "Remediation completed at $(date)" | tee -a /var/log/config-audit.log
```

5. AUDIT TRAILS

- ✓ Keep a record of what changed, when and by whom.
- ✓ Log detection and remediation actions.
- ✓ Store logs locally or in a centralized system.
- ✓ Include old and new values when possible.
- ✓ Useful for compliance, troubleshooting and security investigations.

Example: Logging changes

```
LOG_FILE="/var/log/config-audit.log"

echo "[$(date)] Drift detected in $CONFIG_FILE" | tee -a "$LOG_FILE"
echo "[$(date)] Backup created: $BACKUP_DIR" | tee -a "$LOG_FILE"
echo "[$(date)] Remediation applied" | tee -a "$LOG_FILE"
```

6. WHEN NOT TO AUTO-REMEDiate

- ✓ Do not auto-remediate in critical production systems without proper testing.
- ✓ Avoid auto-remediation if the cause of drift is unknown.
- ✓ Do not overwrite intentional manual changes.
- ✓ Be careful with dynamic configurations that change frequently (e.g. certificates, IP addresses).
- ✓ Use alerts or manual approval for high-risk changes.

7. KEY TAKEAWAYS

- ✓ Understand desired vs actual state.
- ✓ Use checksums and diff to detect drift.
- ✓ Always back up before remediation.
- ✓ Validate configuration after changes.
- ✓ Keep audit trails for all actions.
- ✓ Do not auto-remediate in high-risk situations.



SENIOR ENGINEER TIP

Combine simple drift detection scripts with version control and centralized logging. For large environments, use configuration management tools (e.g. Ansible, Puppet, Chef) to automate detection and safe remediation.



REAL WORLD LESSON

Automatic remediation can fix drift, but it can also hide real problems. Always understand why the drift happened and ensure remediation does not cause downtime or overwrite intentional changes.

16

Automating Process Investigation

ps | pgrep | /proc | process trees | identifying abnormal processes

1. WHY INVESTIGATE PROCESSES?

- ✓ Find what is running on the system.
- ✓ Detect abnormal or unwanted processes.
- ✓ Troubleshoot resource issues (CPU, memory, disk, network).
- ✓ Identify parent/child relationships.
- ✓ Useful for security investigations and automation.

2. USING ps (VIEW PROCESSES)

- ✓ Shows running processes.
- ✓ Provides details like PID, user, CPU, memory, start time.
- ✓ Commonly used with options to filter and format output.

Example: Common ps commands

```
# Show all processes
ps aux
# Show specific columns
ps -eo pid,user,pcpu,pmem,comm --sort=-pcpu | head
# Show processes for a specific user
ps -u nginx -o pid,ppid,cmd
```

3. USING pgrep (FIND PROCESSES)

- ✓ Search for processes by name or pattern.
- ✓ Returns only PIDs (useful for scripting).
- ✓ Supports additional options like parent PID, exact match, oldest/newest.

Example: Using pgrep

```
# Find process IDs by name
pgrep nginx
# Find with full command line match
pgrep -f "nginx.conf"
# Find processes for a user
pgrep -u deploy -f myapp
# Get the oldest matching process
pgrep -n -o python
```

4. EXPLORING /proc (PROCESS INFORMATION)

- ✓ /proc provides detailed information about running processes.
- ✓ Each process has a directory: /proc/<PID>
- ✓ Useful for checking command line, environment, open files and resource usage.

Example: Inspecting /proc

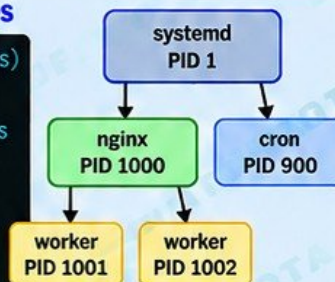
```
# View command line of a process
cat /proc/1234/cmdline | tr '\0' ' '
# View process environment
cat /proc/1234/environ | tr '\0' '\n'
# View resource usage
cat /proc/1234/status
# List open files
ls -l /proc/1234/fd
```

5. PROCESS TREES (PARENT AND CHILD PROCESSES)

- ✓ Processes have parent and child relationships.
- ✓ Helps understand how a process was started.
- ✓ Useful for investigating unexpected processes.
- ✓ Common tools: pstree, ps, pgrep.

Example: Viewing process trees

```
# Show process tree (all processes)
pstree -p
# Show tree for a specific process
pstree -p 1234
# Show parent process of a PID
ps -o ppid= -p 1234
```



6. IDENTIFYING ABNORMAL PROCESSES

- ✓ Look for unknown or unexpected processes.
- ✓ Check high CPU, memory or I/O usage.
- ✓ Verify the process owner and command line.
- ✓ Investigate processes running from unusual locations (e.g. /tmp, /dev/shm).
- ✓ Check for suspicious parent processes.

Example: Finding suspicious processes

```
# Find processes using high CPU (top 5)
ps aux --sort=-%cpu | head -n 5
# Find processes using high memory (top 5)
ps aux --sort=-%mem | head -n 5
# Find processes running from /tmp (suspicious)
ps aux | grep -E '/tmp|/dev/shm'
# Show processes with no valid command name
ps -eo pid,comm | grep '^s*$'
```

7. PRACTICAL EXAMPLE: PROCESS INVESTIGATION SCRIPT

```
#!/bin/bash
# Investigate a process by name
NAME="$1"
echo "Searching for process: $NAME"
echo "Process details:"
ps aux | grep -E "$NAME" | grep -v grep
echo -e "\nProcess tree:"
pgrep -f "$NAME" | while read pid; do
    pstree -p $pid
done
```

8. KEY TAKEAWAYS

- ✓ Use ps to view detailed process info.
- ✓ Use pgrep to find PIDs quickly.
- ✓ Explore /proc for in-depth details.
- ✓ Use process trees to understand relationships.
- ✓ Look for unusual processes, high resource usage and suspicious locations.
- ✓ Automate checks for regular monitoring.



REAL WORLD LESSON

Unexpected or malicious processes can consume resources, exfiltrate data or disrupt services. Regularly monitor running processes and automate detection of abnormal processes to keep your systems secure and stable.



SENIOR ENGINEER TIP

Combine ps, pgrep, /proc and process trees in scripts to create automated process monitoring and alerting for production systems.

17 Safe Process Lifecycle Automation

graceful termination | SIGTERM before SIGKILL | timeout handling | verifying termination

1. WHY SAFE PROCESS LIFECYCLE AUTOMATION?

- ✓ Allows services to shut down cleanly.
- ✓ Prevents data loss and corruption.
- ✓ Gives applications time to finish ongoing work.
- ✓ Handles unresponsive processes with timeouts.
- ✓ Useful for deployments, restarts and automation scripts.

2. UNDERSTANDING TERMINATION SIGNALS

Signal	Number	Purpose	Behavior
SIGTERM	15	Request graceful termination	Lets the process clean up and exit.
SIGKILL	9	Force immediate termination	Does not allow cleanup. Use only if needed.
SIGINT	2	Interrupt from keyboard (Ctrl+C)	Graceful termination for many applications.
SIGHUP	1	Reload configuration (or terminate)	Used for config reload in many services.

3. GRACEFUL TERMINATION (SIGTERM FIRST)

- ✓ Send SIGTERM to request a clean shutdown.
- ✓ Give the process time to exit.
- ✓ Check if it is still running.
- ✓ If still running, use SIGKILL.
- ✓ Always verify that the process has terminated.

Example: Graceful termination script

```
#!/bin/bash
PID=$1
if [ -z "$PID" ]; then
    echo "Usage: $0 <PID>"
    exit 1
fi
echo "Sending SIGTERM to process $PID ..."
kill -TERM "$PID"
sleep 10
if kill -0 "$PID" 2>/dev/null; then
    echo "Process still running. Sending SIGKILL ..."
    kill -KILL "$PID"
else
    echo "Process terminated gracefully."
fi
```

4. TIMEOUT HANDLING

- ✓ Use a timeout to wait for graceful shutdown.
- ✓ If the process does not exit within the timeout, use SIGKILL.
- ✓ The timeout period depends on the application.
- ✓ Useful in automation, deployment and maintenance scripts.

Example: Using timeout with verification

```
#!/bin/bash
PID=$1
TIMEOUT=20
echo "Sending SIGTERM to $PID ..."
kill -TERM "$PID"
echo "Waiting for $TIMEOUT seconds ..."
for i in $(seq 1 $TIMEOUT); do
    if ! kill -0 "$PID" 2>/dev/null; then
        echo "Process terminated."
        exit 0
    fi
    sleep 1
done
echo "Process did not terminate. Sending SIGKILL ..."
kill -KILL "$PID"
```

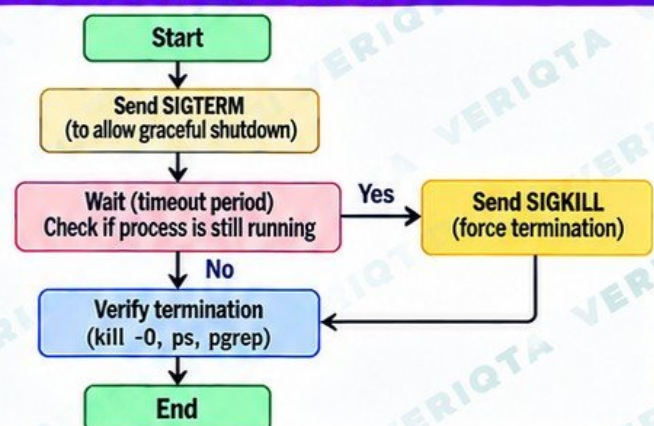
5. VERIFYING TERMINATION

- ✓ Use kill -0 to check if a process is still running.
- ✓ Use ps or pgrep to confirm.
- ✓ Check logs to ensure clean shutdown.
- ✓ Verify that dependent processes are also terminated.

Example: Verify process status

```
# Check if process is running
if kill -0 "$PID" 2>/dev/null; then
    echo "Process is still running."
    ps -p "$PID" -o pid,ppid,cmd
else
    echo "Process is not running."
fi
# Or search by name
pgrep -a nginx
```

6. PROCESS TERMINATION FLOW



7. KEY TAKEAWAYS

- ✓ Always try SIGTERM before SIGKILL.
- ✓ Use timeout handling in automation scripts.
- ✓ Verify that the process has terminated.
- ✓ Check logs for a clean shutdown.
- ✓ Handle dependent child processes if needed.
- ✓ Adjust timeout values based on the application.



SENIOR ENGINEER TIP

Use graceful termination in all automation and deployment scripts. It reduces the risk of data loss, keeps services stable and makes your automation more reliable.



REAL WORLD LESSON

Forcing a process to stop with SIGKILL can cause data loss, corrupted files or inconsistent state. Always give the application time to shut down gracefully before using SIGKILL.

18 Automating systemd Services

units | dependencies | restart policies | daemon reloads | service verification

1. WHAT IS SYSTEMD?

- ✓ Systemd is the default init system on most modern Linux distributions.
- ✓ It manages services, sockets, timers and more.
- ✓ Uses unit files to define how a service runs.
- ✓ Provides dependency management, automatic restarts and better process supervision.
- ✓ Essential for production automation and reliability.

2. SYSTEMD UNIT STRUCTURE

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My Application Service
After=network.target
Wants=network-online.target

[Service]
Type=simple
User=myuser
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/python3 app.py
Restart=on-failure
RestartSec=5
[Install]
WantedBy=multi-user.target
```

Unit information and dependencies

How the service runs

Restart policy and settings

When to start the service

3. MANAGING SERVICES (BASIC COMMANDS)

- ✓ Start, stop, restart and enable services.
- ✓ Check status and view logs.
- ✓ Reload systemd when unit files change.
- ✓ Essential commands for automation scripts.

Common commands

```
# Reload systemd after creating or editing a unit file
sudo systemctl daemon-reload
# Start a service
sudo systemctl start myapp
# Stop a service
sudo systemctl stop myapp
# Restart a service
sudo systemctl restart myapp
# Enable service at boot
sudo systemctl enable myapp
# Check service status
sudo systemctl status myapp
# View logs
sudo journalctl -u myapp -f
```

4. DEPENDENCIES

- ✓ Use After= to start after another unit.
- ✓ Use Requires= or Wants= to handle dependencies.
- ✓ Common targets: network.target, network-online.target, local-fs.target.
- ✓ Helps ensure services start in the correct order.

Example: Service with dependencies

```
[Unit]
Description=My Application
After=network-online.target mysql.service
Wants=network-online.target
Requires=mysql.service
```

5. RESTART POLICIES

- ✓ Restart services automatically on failure.
- ✓ Common values: no, on-failure, always, unless-stopped.
- ✓ Use RestartSec= to set delay between restarts.
- ✓ Useful for high-availability and production systems.

Example: Restart policy

```
[Service]
ExecStart=/usr/bin/python3 app.py
Restart=on-failure
RestartSec=5
StartLimitIntervalSec=60
StartLimitBurst=3
```

6. DAEMON RELOADS

- ✓ Run daemon-reload after creating or modifying unit files.
- ✓ Tells systemd to read the updated configuration.
- ✓ Does not restart the service, only reloads unit definitions.

Example: Reload systemd and restart service

```
sudo systemctl daemon-reload
sudo systemctl restart myapp
sudo systemctl status myapp
```

7. SERVICE VERIFICATION

- ✓ Check service is active and running.
- ✓ Verify it starts at boot.
- ✓ Check logs for errors.
- ✓ Confirm correct process is running.

Example: Verify service

```
sudo systemctl is-active myapp
sudo systemctl is-enabled myapp
systemctl status myapp --no-pager
ps -ef | grep myapp
journalctl -u myapp --since "1 hour ago"
```

8. AUTOMATION EXAMPLE: CREATE, ENABLE AND VERIFY

```
#!/bin/bash
SERVICE_NAME="myapp"
UNIT_FILE="/etc/systemd/system/${SERVICE_NAME}.service"

# Create unit file
sudo tee $UNIT_FILE <<EOF
[Unit]
Description=My Application
After=network-online.target

[Service]
ExecStart=/usr/bin/python3 /opt/myapp/app.py
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF
# Reload, enable and start
sudo systemctl daemon-reload
sudo systemctl enable $SERVICE_NAME
sudo systemctl start $SERVICE_NAME
sudo systemctl status $SERVICE_NAME --no-pager
```

9. KEY TAKEAWAYS

- ✓ Use unit files to define services.
- ✓ Set correct dependencies.
- ✓ Configure restart policies.
- ✓ Always run daemon-reload after changes.
- ✓ Verify service status and logs.
- ✓ Automate with scripts for consistent deployments.



REAL WORLD LESSON

Incorrect dependencies or restart policies can cause services to fail at boot, enter restart loops or hide underlying issues. Always verify service status and logs after automation.



SENIOR ENGINEER TIP

Use systemd unit templates and drop-in files for flexible and maintainable automation.

19 systemd Timers Versus Cron

scheduling models | missed executions | dependencies | logging | operational visibility

1. OVERVIEW

- Both cron and systemd timers can schedule commands and scripts.
- They use different scheduling models.
- Systemd timers provide better reliability, logging and operational visibility.
- Systemd timers can handle missed executions and support dependencies.
- Cron is simple and widely available, but has limitations for complex automation.

2. CRON VS SYSTEMD TIMERS

Feature	Cron	systemd Timers
Scheduling model	Time based (cron expression)	Time based or event based (calendar or monotonic)
Missed executions	Skipped if system is down	Can catch up with Persistent=true
Dependencies	No built-in dependencies	Can define dependencies with After= and Requires=
Logging	Usually in system logs or custom log files	Integrated with journald (journalctl)
Operational visibility	Limited	Detailed status, history and next run time
Configuration	Simple crontab file	Unit and timer files
Best use case	Simple repetitive tasks	Reliable, production ready automation

3. CRON SCHEDULING MODEL

- Uses a 5-field time expression (minute hour day month day-of-week).
- Runs the command at the specified time.
- Does not run missed jobs if the system is down.
- No built-in support for dependencies.
- Output must be redirected to a log file for visibility.

Example: Cron job

```
# Edit crontab
crontab -e

# Run backup script every day at 2:00 AM
0 2 * * * /usr/local/bin/backup.sh >> /var/log/backup.log 2>&1
```

4. SYSTEMD TIMER SCHEDULING MODEL

- Uses calendar based schedules (e.g. daily, hourly, specific time).
- Can catch up missed executions with Persistent=true.
- Supports dependencies with After=, Requires=.
- Logs to journald (view with journalctl).
- Provides detailed status, next run time and history.

Example: systemd timer files

```
# /etc/systemd/system/backup.service
[Unit]
Description=Backup Script
[Service]
Type=oneshot
ExecStart=/usr/local/bin/backup.sh

# /etc/systemd/system/backup.timer
[Unit]
Description=Run backup daily
[Timer]
OnCalendar=daily 02:00
Persistent=true
[Install]
WantedBy=timers.target
```

5. HANDLING MISSED EXECUTIONS

- Cron: missed runs are skipped if the system was down at the scheduled time.
- systemd timers: use Persistent=true to run the job immediately after the system comes back up.

Example: systemd timer with Persistent=true

```
[Timer]
OnCalendar=daily 02:00
Persistent=true
[Install]
WantedBy=timers.target
```

6. DEPENDENCIES

- Cron does not have built-in dependency management.
- systemd timers can define dependencies using After=, Requires=.
- Useful when a job depends on network, database or another service.

Example: timer with dependency on network

```
[Unit]
Description=Run backup after network is online
After=network-online.target
Requires=network-online.target
[Timer]
OnCalendar=daily 02:00
Persistent=true
```

7. LOGGING AND OPERATIONAL VISIBILITY

- Cron: output must be redirected to a file or rely on system logs.
- systemd timers: logs are sent to journald automatically.
- You can easily view past runs, status and next execution time.

Example: View timer status and logs

```
# Check timer status
systemctl status backup.timer

# View last runs
journalctl -u backup.service

# See next scheduled run
systemctl list-timers backup.timer
```

9. KEY TAKEAWAYS

- Understand the scheduling models.
- Use systemd timers for reliable, production automation.
- Enable Persistent=true to handle missed executions.
- Use dependencies for correct execution order.
- Leverage journald for better logging and visibility.

10. REAL WORLD USE CASES

- Automated backups
- Log rotation
- Health checks
- Report generation
- Service maintenance tasks

8. PRACTICAL COMPARISON



SENIOR ENGINEER TIP
Use systemd timers for critical production jobs to improve reliability, visibility and operational control. Keep cron for simple tasks or legacy systems.

20

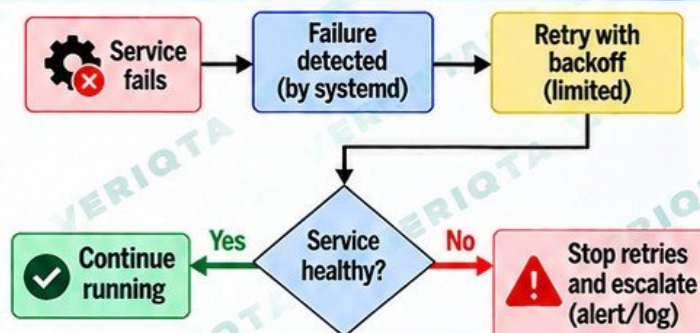
Self-Healing Services Without Restart Loops

failure detection | bounded retries | backoff | restart limits | escalation

1. WHAT DOES SELF-HEALING MEAN?

- ✓ Automatically detects when a service fails.
- ✓ Attempts to recover using controlled retries.
- ✓ Prevents infinite restart loops.
- ✓ Escalates when the issue cannot be resolved automatically.
- ✓ Keeps services reliable and avoids unnecessary downtime.

2. HOW IT WORKS



3. FAILURE DETECTION

- ✓ Systemd detects failures based on exit codes, timeouts or health checks.
- ✓ Common failure causes: application crash, resource exhaustion, dependency failure, unresponsive service.
- ✓ Use proper ExecStart, ExecStartPre or a health check script to validate the service.

Example: Basic service with failure detection

```

[Unit]
Description=My Application
After=network-online.target

[Service]
Type=simple
ExecStart=/usr/bin/python3 /opt/myapp/app.py
Restart=on-failure
RestartSec=10
StartLimitIntervalSec=300
StartLimitBurst=5

[Install]
WantedBy=multi-user.target
  
```

4. BOUNDED RETRIES AND BACKOFF

- ✓ Do not restart immediately without limits.
- ✓ Use Restart=on-failure to retry only on failure.
- ✓ Use RestartSec to add a delay between retries.
- ✓ Use StartLimitIntervalSec and StartLimitBurst to prevent infinite restart loops.
- ✓ Consider exponential backoff for custom scripts.

Example: Service with backoff and limits

```

[Service]
Type=notify
ExecStart=/opt/myapp/start.sh
Restart=on-failure
RestartSec=30 # wait 30 seconds before retry
StartLimitIntervalSec=300 # 5 minutes
StartLimitBurst=3 # max 3 restarts in 5 minutes
  
```

5. ESCALATION WHEN RECOVERY FAILS

- ✓ Stop retrying after the limit is reached.
- ✓ Log the failure for investigation.
- ✓ Send an alert (email, Slack, PagerDuty, etc).
- ✓ Optionally run a custom script for escalation.

Example: Escalation script (executed after failure)

```

#!/bin/bash
SERVICE_NAME="myapp"
echo "$(date): $SERVICE_NAME failed multiple times."

# Send alert (example using mail)
echo "Service $SERVICE_NAME is down. Check logs." | \
mail -s "ALERT: $SERVICE_NAME Failure" admin@example.com
  
```

6. VERIFYING SERVICE STATUS

- ✓ Check if the service is running and healthy.
- ✓ Review restart count and recent logs.
- ✓ Confirm that it is not stuck in a restart loop.
- ✓ Use systemctl, journalctl and systemd status output.

Example: Verification commands

```

# Check service status
systemctl status myapp

# View recent logs
journalctl -u myapp -n 50 --no-pager

# Check restart count
systemctl show myapp -p NRestarts

# Check if service is in a failed state
systemctl is-failed myapp
  
```

7. KEY TAKEAWAYS

- ✓ Use failure detection and health checks.
- ✓ Apply bounded retries with backoff.
- ✓ Set restart limits to prevent loops.
- ✓ Log and monitor all failures.
- ✓ Escalate when automatic recovery fails.

8. REAL WORLD USE CASES

- ✓ Web application auto-recovery
- ✓ API service with dependency checks
- ✓ Worker service with backoff and limits
- ✓ Database client with health check
- ✓ Alerting on repeated failures

9. SENIOR ENGINEER TIP

- 💡 Combine systemd restart limits with application-level health checks and external monitoring (e.g. Prometheus, Grafana, PagerDuty). This prevents restart loops and ensures issues are escalated quickly.

1. WHY DISK CAPACITY PROTECTION?

- ✓ Prevents services from failing due to full disks.
- ✓ Avoids application errors and data loss.
- ✓ Detects both disk space and inode exhaustion.
- ✓ Uses thresholds and alerts before it becomes critical.
- ✓ Automates safe cleanup to maintain system health.

2. KEY CONCEPTS



Disk space

Monitored with df

Tracks used and available space by filesystem.



File usage

Monitored with du

Finds large files and directories.



Inodes

Monitored with df -i

Tracks number of files. Can be exhausted even if disk space is free.



Thresholds

Set warning and critical limits (e.g. 70%, 85%, 95%).



Safe cleanup policies

Remove only safe files (logs, caches, temp files) based on defined rules.

3. CHECKING DISK USAGE WITH df

- ✓ Shows filesystem disk space usage.
- ✓ Use -h for human readable output.
- ✓ Monitor important filesystems (/ , /var , /home , /tmp).
- ✓ Set thresholds to alert before disk is full.

Example: Check disk usage

```
# Show disk usage (human readable)
df -h
# Check specific filesystem
df -h /var
```

4. FINDING LARGE FILES WITH du

- ✓ Identifies directories and files using the most space.
- ✓ Helps locate what is consuming disk space.
- ✓ Use -h for human readable sizes.
- ✓ Combine with sort to find largest items.

Example: Find large directories

```
# Show top 10 largest directories in /var
du -sh /var/* | sort -h | tail -n 10
# Find large files (top 10)
du -ah /var | sort -rh | head -n 10
```

5. CHECKING INODE USAGE

- ✓ Inodes represent files and directories.
- ✓ You can run out of inodes even if disk space is free.
- ✓ Use df -i to check inode usage.
- ✓ Monitor and alert on high inode usage.

Example: Check inode usage

```
# Show inode usage
df -i
# Check specific filesystem
df -i /var
```

6. THRESHOLDS AND ALERTING

- ✓ Set warning and critical thresholds for disk space and inodes.
- ✓ Monitor key filesystems (/ , /var , /home , /tmp , /var/log).
- ✓ Send alerts via email, Slack or monitoring tools.
- ✓ Helps you act before services are affected.

Example: Check threshold with script

```
#!/bin/bash
THRESHOLD=85 # 85%
USAGE=$(df -h / | awk 'NR==2 {print $5}' | tr -d '%')
if [ "$USAGE" -ge "$THRESHOLD" ]; then
    echo "Disk usage is ${USAGE}%. Alert!"
fi
```

7. SAFE CLEANUP POLICIES

- ✓ Only remove safe files (logs, caches, temp files).
- ✓ Do not delete application data or important files.
- ✓ Use defined rules and retention policies.
- ✓ Always test cleanup commands before automating.

Example: Cleanup old log files (safe)

```
# Remove log files older than 7 days
find /var/log -type f -name "*.log" -mtime +7 -delete
# Remove old package caches (Debian/Ubuntu)
sudo apt clean
# Remove temporary files older than 7 days
find /tmp -type f -mtime +7 -delete
```

8. AUTOMATION EXAMPLE

- ✓ Check disk space and inode usage.
- ✓ Alert if thresholds are exceeded.
- ✓ Clean up only safe files.
- ✓ Log all actions for audit and troubleshooting.

Example: Simple disk monitoring script

```
#!/bin/bash
# Check disk usage
df -h | awk 'NR>1 {print $5, $6}' | while read use mount; do
    use=${use%%%}
    if [ "$use" -ge 85 ]; then
        echo "$(date) WARNING: $mount is ${use}% full" | \
            tee -a /var/log/disk-monitor.log
    fi
done
```

9. KEY TAKEAWAYS

- ✓ Use df to monitor disk space.
- ✓ Use du to find large files and directories.
- ✓ Monitor inode usage with df -i.
- ✓ Set thresholds and send alerts.
- ✓ Use safe cleanup policies and automation.

10. REAL WORLD USE CASES

- ✓ Prevent application downtime due to full disk.
- ✓ Monitor /var/log on production servers.
- ✓ Detect inode exhaustion on web servers.
- ✓ Automate cleanup of old logs and temp files.
- ✓ Alert before disk or inode usage becomes critical.

SENIOR ENGINEER TIP



Monitor both disk space and inodes. Many outages are caused by inode exhaustion, not disk space. Combine monitoring, alerting and automated cleanup for a resilient system.

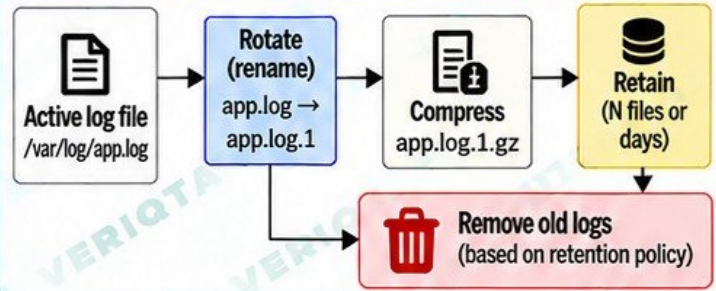
22 Automated Log Rotation and Retention

logrotate | compression | retention | disk protection | audit implications

1. WHY LOG ROTATION AND RETENTION?

- ✓ Prevents disks from filling up due to large log files.
- ✓ Keeps logs manageable and easier to analyse.
- ✓ Automatically compresses and removes old logs.
- ✓ Protects system performance and availability.
- ✓ Helps meet audit, compliance and legal requirements.

2. HOW LOGROTATE WORKS



3. BASIC LOGROTATE CONFIGURATION

- ✓ Define which logs to rotate.
- ✓ Set rotation frequency (daily, weekly, etc).
- ✓ Configure retention (number of files or days).
- ✓ Enable compression to save disk space.
- ✓ Optionally run a script after rotation.

Example: /etc/logrotate.d/myapp

```

/var/log/myapp/*.log {
    daily
    rotate 7
    compress
    missingok
    notifempty
    create 0640 appuser appgroup
    postrotate
        systemctl reload myapp >/dev/null 2>&1 || true
    endscript
}
  
```

4. KEY DIRECTIVES

Directive	Purpose
daily / weekly / monthly	Set rotation frequency.
rotate N	Number of old logs to keep.
compress	Compress rotated logs (uses gzip).
delaycompress	Compress from the next rotation.
missingok	Do not error if log file is missing.
notifempty	Do not rotate empty log files.
create MODE USER GROUP	Create a new log file after rotation.
postrotate / endscript	Run commands after rotation (e.g. reload a service).

5. COMPRESSION AND RETENTION

- ✓ Compression reduces disk usage (usually gzip).
- ✓ Set an appropriate retention period based on requirements (e.g. 7, 14 or 30 days).
- ✓ Balance between disk space and audit needs.
- ✓ Use delaycompress if applications keep writing after rotation.

6. MANUAL TESTING AND VERIFICATION

- ✓ Test configuration without making changes.
- ✓ Force rotation to verify behaviour.
- ✓ Check that old logs are compressed and retained correctly.

Example: Test logrotate

```

# Check configuration
sudo logrotate -d /etc/logrotate.d/myapp

# Force rotation
sudo logrotate -f /etc/logrotate.d/myapp

# List log files
ls -lh /var/log/myapp/
  
```

7. DISK PROTECTION

- ✓ Log rotation prevents /var from filling up.
- ✓ Use compression to reduce disk usage.
- ✓ Monitor disk usage and set alerts.
- ✓ Combine with retention policies to automatically remove old logs.

8. AUDIT IMPLICATIONS

- ✓ Logs may be required for compliance (e.g. 7, 30 or 90 days).
- ✓ Do not set retention too low if logs are needed for investigations.
- ✓ Ensure log rotation does not delete logs required by policy.
- ✓ Keep audit logs separate with longer retention if required.

9. KEY TAKEAWAYS

- ✓ Use logrotate to automatically manage log files.
- ✓ Always enable compression.
- ✓ Set retention based on operational and audit needs.
- ✓ Protect disk space with proper rotation policies.
- ✓ Test and monitor log rotation regularly.

10. REAL WORLD USE CASES

- ✓ Web server logs (Nginx, Apache).
- ✓ Application logs (e.g. /var/log/myapp/).
- ✓ System logs (e.g. /var/log/syslog).
- ✓ Database logs.
- ✓ Security and audit logs (with longer retention).

SENIOR ENGINEER TIP

- 💡 Set different retention policies for application logs, system logs and audit logs based on their importance and compliance requirements.

/proc/meminfo | swap | OOM evidence | memory thresholds | why automated process killing is dangerous

1. UNDERSTANDING MEMORY PRESSURE

- ✓ Memory pressure happens when the system is running low on available memory.
- ✓ It can cause application slowdowns, failures or Out Of Memory (OOM) kills.
- ✓ Monitor memory usage, swap and key metrics before it becomes critical.
- ✓ Use thresholds and alerts to detect memory pressure early.
- ✓ Be careful with automated process killing (see section 6).

2. CHECKING MEMORY USAGE

```
# View memory information
cat /proc/meminfo

# Human readable summary
free -h

# Detailed memory usage
free -m

# Real time memory usage
watch -n 2 free -h
```

Detailed memory information from the kernel

Summary of total, used, free and available memory

Monitor memory usage in real time

3. KEY /proc/meminfo METRICS

Metric	Description
MemTotal	Total physical memory.
MemAvailable	Estimated available memory (for new applications).
Buffers / Cached	Memory used for caching (can be reclaimed).
SwapTotal	Total swap space.
SwapFree	Free swap space.
Dirty	Memory waiting to be written to disk.
Committed_AS	Total committed memory (RAM + swap).

4. MONITORING SWAP USAGE

- ✓ Swap is used when physical memory is low.
- ✓ High swap usage indicates memory pressure.
- ✓ Continually high swap usage can degrade performance.
- ✓ Monitor swap usage and set alerts.

Example: Check swap usage

```
# View swap usage
free -h | grep Swap

# Show active swap usage
swapon --show

# Detailed swap information
cat /proc/swaps
```

5. DETECTING OOM EVENTS

- ✓ The kernel kills processes when memory is exhausted (Out Of Memory - OOM).
- ✓ Check system logs for OOM evidence.
- ✓ Identify which process was killed and why.
- ✓ Use this information to adjust memory limits and alerts.

Example: Find OOM events in logs

```
# Search for OOM killer messages
sudo journalctl -k | grep -i 'oom'

# Check system logs
sudo dmesg | grep -i 'killed process'

# View last 50 OOM related log entries
sudo journalctl -k -n 50 | grep -i 'oom'
```

6. MEMORY THRESHOLDS AND ALERTS

- ✓ Set warning and critical thresholds based on your environment.
- ✓ Monitor MemAvailable, swap usage and overall memory usage.
- ✓ Send alerts via email, Slack or monitoring tools.
- ✓ Take automated action (e.g. cleanup) before the system runs out of memory.

Example: Simple memory threshold check

```
#!/bin/bash
THRESHOLD=20 # 20% available memory
AVAILABLE=$(free | awk '/MemAvailable/ {printf "%.0f", $7*100/$2}')
if [ "$AVAILABLE" -lt "$THRESHOLD" ]; then
    echo "Warning: Low memory ($(AVAILABLE)% available)"
    # Send alert or run cleanup script here
fi
```

7. WHY AUTOMATED PROCESS KILLING IS DANGEROUS

- ✓ Killing the wrong process can cause application outages.
- ✓ It may hide the real problem (memory leak or misconfiguration).
- ✓ Critical system services might be terminated.
- ✓ Can create a restart loop and make the situation worse.
- ✓ It is better to investigate, alert and take controlled action (e.g. cleanup, increase memory, fix the root cause).

9. KEY TAKEAWAYS

- ✓ Monitor memory, swap and OOM events.
- ✓ Set appropriate thresholds and alerts.
- ✓ Avoid aggressive automated process killing.
- ✓ Take safe, controlled actions before memory runs out.
- ✓ Investigate and fix the root cause.

10. REAL WORLD USE CASES

- ✓ Web server under high traffic (Nginx, Apache).
- ✓ Application memory leak.
- ✓ Database running out of memory.
- ✓ Container or Kubernetes node pressure.
- ✓ System with limited RAM (e.g. cloud instances).

8. SAFE AUTOMATION ACTIONS

- ✓ Clean up temporary files and logs.
- ✓ Remove unused cache or old containers.
- ✓ Restart non-critical services (only when safe).
- ✓ Increase swap or memory if possible.
- ✓ Always log what actions were taken.
- ✓ Combine with monitoring and alerting for visibility.

SENIOR ENGINEER TIP



Use monitoring tools (Prometheus, Grafana, Datadog) to track trends, set alerts and identify memory leaks early. Always investigate OOM events and fix the root cause instead of relying on automated process killing.

load average | CPU saturation | run queues | uptime | vmstat | avoiding misleading thresholds

1. UNDERSTANDING CPU AND LOAD

- ✓ CPU and load metrics help you understand system performance and detect issues early.
- ✓ Load average shows the number of processes waiting for CPU (not CPU usage percentage).
- ✓ High CPU usage can indicate CPU saturation.
- ✓ Run queues show how many processes are waiting to run.
- ✓ Use uptime, vmstat and other tools to get a complete picture.

2. KEY CONCEPTS

Concept	Description
Load average	Average number of processes in the run queue (running or waiting for CPU).
CPU saturation	When CPU is fully utilized and processes have to wait.
Run queues	Number of processes waiting for CPU time.
Uptime	Shows how long the system has been running and current load averages.
vmstat	Provides CPU, memory, IO and run queue information.
Misleading thresholds	Static thresholds (e.g. load > 1) can be misleading. Consider CPU cores, workload type and baseline metrics.

3. CHECKING LOAD AVERAGE AND UPTIME

- ✓ Use uptime to view load averages.
- ✓ Load average has 3 values: 1, 5 and 15 minutes.
- ✓ Compare load average with the number of CPU cores.
- ✓ A high load average indicates CPU or IO pressure.

Example: Check load average and uptime

```
# View uptime and load average
uptime
10:24:01 up 5 days, 3:12, 2 users, load average:
0.35, 0.42, 0.50

# Show number of CPU cores
nproc
# or
lscpu | grep -E "CPU(s):"
```

4. CHECKING CPU USAGE AND RUN QUEUES

- ✓ Use top or htop to view real-time CPU usage.
- ✓ Use vmstat to check run queue, CPU, memory and IO.
- ✓ High run queue values indicate processes waiting for CPU.

Example: Using top and vmstat

```
# Real-time CPU usage
top -bn1 | head -20

# Check run queue and CPU statistics (every 2 seconds)
vmstat 2 5

procs  -----memory-----  ---swap--  -----io-----  -system--  ----cpu----
r  b   swpd  free  buff  cache  si   so    bi   bo    in  cs  us  sy  id  wa  st
1  0     0  826112 123456 2345676 0    0   12    8   150 300 25  5 70  0  0
3  0     0  824000 123400 2345000 0    0   10    6   140 280 80 10 10  0  0
5  2     0  823800 123300 2344000 0    0   20   10   300 500 95  4  1  0  0
```

5. INTERPRETING LOAD AVERAGE

- ✓ Load average should be compared to the number of CPU cores.
- ✓ A load average of 1.0 per core is generally considered high for sustained periods.
- ✓ Short spikes are normal, but sustained high values indicate a problem.

Example: Check load average against CPU cores

```
# Get CPU cores and load average
CORES=$(nproc)
LOAD=$(uptime | awk -F'load average:' '{print $2}' | \
    awk -F',' '{print $1}')
echo "CPU cores: $CORES"
echo "Load average (1 min): $LOAD"
```

6. AVOIDING MISLEADING THRESHOLDS

- ✓ Do not use static thresholds without considering CPU cores.
- ✓ Consider workload type (CPU intensive, IO intensive, etc).
- ✓ Use baseline metrics and trends.
- ✓ Monitor run queue, CPU usage, IO wait and context switches.

Example: Better threshold approach (Bash script)

```
#!/bin/bash
# Alert if load average is higher than number of CPU cores
CORES=$(nproc)
LOAD=$(uptime | awk -F'load average:' '{print $2}' | \
    awk -F',' '{print $1}')
if (( $(echo "$LOAD > $CORES" | bc -l) )); then
    echo "WARNING: Load average ($LOAD) is higher than \
        "CPU cores ($CORES)"
    # Send alert here (email, Slack, etc.)
fi
```

7. KEY TAKEAWAYS

- ✓ Load average shows processes waiting for CPU.
- ✓ Compare load with the number of CPU cores.
- ✓ Use vmstat to check run queues and CPU usage.
- ✓ Avoid static thresholds.
- ✓ Monitor trends and use multiple metrics.

8. REAL WORLD USE CASES

- ✓ Detecting CPU saturation before performance issues affect users.
- ✓ Monitoring high run queues in production.
- ✓ Identifying runaway processes.
- ✓ Capacity planning and scaling decisions.
- ✓ Automated alerts based on load and run queue metrics.

9. SENIOR ENGINEER TIP



Use multiple metrics together (load average, CPU usage, run queue, IO wait, context switches) and baseline values for your specific environment. This reduces false alerts and helps detect real issues early.

25

File Descriptor and Resource Limit Monitoring

ulimit | /proc/<pid>/fd | sysctl | exhaustion detection | safe remediation

1. WHY MONITOR FILE DESCRIPTORS AND LIMITS?

- ✓ Prevents application failures due to file descriptor exhaustion.
- ✓ Helps detect resource limit issues early.
- ✓ Ensures system stability and performance.
- ✓ Identifies leaks and abnormal usage patterns.
- ✓ Allows safe and controlled remediation before services are affected.

2. KEY CONCEPTS

Concept	Description
File Descriptor (FD)	A handle to an open file, socket or resource (every process has a limit).
ulimit	Shows or sets per-user and per-process limits.
/proc/<pid>/fd	Lists the open file descriptors for a process.
sysctl	Kernel parameters that control system-wide limits (e.g. fs.file-max).
Exhaustion detection	Monitoring FD usage to detect when limits are close to being reached.
Safe remediation	Increasing limits, fixing leaks or optimizing applications.

3. CHECKING RESOURCE LIMITS WITH ulimit

- ✓ Use ulimit -n to check the maximum number of open files.
- ✓ Use ulimit -a to view all resource limits.
- ✓ Compare soft and hard limits.
- ✓ Adjust limits when needed (per user, service or system).

Example: Check and set file descriptor limits

```
# Check current limits
ulimit -n
ulimit -a

# Set a higher limit (temporary)
ulimit -n 65535

# Set both soft and hard limits (in /etc/security/limits.conf)
echo "* soft nofile 65535" | sudo tee -a /etc/security/limits.conf
echo "* hard nofile 65535" | sudo tee -a /etc/security/limits.conf
```

4. MONITORING OPEN FILE DESCRIPTORS

- ✓ Use /proc/<pid>/fd to see open file descriptors.
- ✓ Check the number of FDs used by a process.
- ✓ Identify unusual growth or leaks.
- ✓ Investigate which files or sockets are open.

Example: Inspect open file descriptors

```
# List open file descriptors for a process
ls -l /proc/$(pgrep -n nginx)/fd

# Count number of open file descriptors
ls /proc/$(pgrep -n nginx)/fd | wc -l

# Show top processes by open files
lsof -n | awk '{print $1}' | sort | uniq -c | \
sort -nr | head -n 10
```

5. SYSTEM-WIDE LIMITS WITH sysctl

- ✓ Use sysctl to view and set kernel-level limits.
- ✓ Important parameters: fs.file-max, fs.nr_open.
- ✓ Ensure system-wide limits are high enough for workloads.
- ✓ Make changes persistent in /etc/sysctl.conf.

Example: Check and configure system limits

```
# Check current system limits
sysctl fs.file-max
sysctl fs.nr_open

# Increase the limits (temporary)
sudo sysctl -w fs.file-max=2097152
sudo sysctl -w fs.nr_open=2097152

# Make it permanent (in /etc/sysctl.conf)
echo "fs.file-max=2097152" | sudo tee -a /etc/sysctl.conf
echo "fs.nr_open=2097152" | sudo tee -a /etc/sysctl.conf
```

6. EXHAUSTION DETECTION

- ✓ Monitor FD usage against soft and hard limits.
- ✓ Look for processes approaching or exceeding limits.
- ✓ Set alerts when usage crosses defined thresholds.
- ✓ Check for signs of leaks (continuous growth over time).

Example: Detect processes close to FD limit

```
# Show processes using high number of file descriptors
lsof -n | awk '{print $1}' | sort | uniq -c | \
sort -nr | head -n 10

# Check a specific process against its limit
PID=$(pgrep -n myapp)
USED=$(ls /proc/$PID/fd | wc -l)
LIMIT=$(cat /proc/$PID/limits | grep "Max open files" | head -1)
echo "Process: $PID"
echo "Open FDs: $USED"
echo "Limit: $LIMIT"
```

7. SAFE REMEDIATION

- ✓ Increase limits gradually based on workload needs.
- ✓ Fix file descriptor leaks in applications.
- ✓ Close unnecessary files and connections.
- ✓ Use monitoring and alerts to act early.
- ✓ Test changes in a non-production environment first.

8. REAL WORLD USE CASES

- ✓ Web server handling many client connections.
- ✓ Database applications with high concurrent users.
- ✓ Microservices and containers with many sockets.
- ✓ Detecting and fixing file descriptor leaks.
- ✓ Preventing outages due to FD exhaustion.

9. SENIOR ENGINEER TIP



Monitor both per-process limits (ulimit) and system-wide limits (sysctl). Set alerts on FD usage trends, not just static thresholds. Investigate growth early to catch leaks before they cause outages.



VERIQTA



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

1. WHY AUTOMATE NETWORK HEALTH CHECKS?

- ✓ Detect network issues before they impact users.
- ✓ Verify critical dependencies (DNS, routing, ports, services).
- ✓ Identify partial failures (e.g., DNS works but HTTP fails).
- ✓ Provide early alerts and automatic actions.
- ✓ Improve service reliability and incident response time.

2. KEY CONCEPTS

Concept	Description
DNS	Resolves domain names to IP addresses.
Routing	Ensures the correct network path to a destination.
Ports	Checks if a specific port is open and reachable.
TCP	Verifies TCP connectivity and handshake.
HTTP	Checks application endpoints (status codes, response time).
Dependency-aware checks	Validates multiple related services in the correct order (e.g., DNS → network → port → application).

3. CHECKING DNS RESOLUTION

- ✓ Verify domain name resolution.
- ✓ Check DNS response time and correct IP address.
- ✓ Test multiple DNS servers if needed.

Example: Check DNS resolution

```
# Resolve a domain name
dig +short example.com

# Check using a specific DNS server
dig @8.8.8.8 example.com

# Measure response time
time dig example.com
```

4. CHECKING ROUTING AND NETWORK PATH

- ✓ Verify network route to the destination.
- ✓ Identify latency and packet loss.
- ✓ Helps detect routing issues or network congestion.

Example: Check network routing

```
# Show route to a host
ip route get example.com

# Trace the network path
traceroute example.com

# Check latency and packet loss
mtr -rw example.com
```

5. CHECKING PORTS AND TCP CONNECTIVITY

- ✓ Verify if a port is open and accepting connections.
- ✓ Check TCP handshake and response time.
- ✓ Useful for database, web and other critical services.

Example: Check port and TCP connectivity

```
# Check if a port is open
nc -zv example.com 80

# Use telnet to test TCP connection
telnet example.com 443

# Show open ports on a host
nmap -Pn example.com
```

6. CHECKING HTTP SERVICES

- ✓ Verify application endpoints are reachable.
- ✓ Check HTTP status codes and response time.
- ✓ Validate content if needed (e.g., expected text).

Example: Check HTTP endpoint

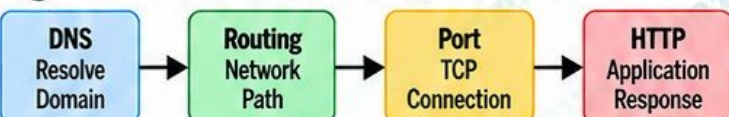
```
# Check HTTP status code
curl -I https://example.com

# Check response time
curl -o /dev/null -s -w "Time: %{time_total}s\n" \
https://example.com

# Check for specific content
curl -s https://example.com | grep -i "welcome"
```

7. DEPENDENCY-AWARE CHECKS

- ✓ Check dependencies in the correct order.
- ✓ Validate that all required services are healthy.
- ✓ Avoid false alarms by considering dependencies.



9. KEY TAKEAWAYS

- ✓ Monitor DNS, routing, ports, TCP and HTTP.
- ✓ Use dependency-aware checks.
- ✓ Set thresholds and alerts.
- ✓ Automate checks for early detection.
- ✓ Integrate with monitoring and incident response.

10. REAL WORLD USE CASES

- ✓ Monitoring website availability.
- ✓ Checking database connectivity.
- ✓ Validating third-party APIs.
- ✓ Detecting DNS or routing issues.
- ✓ Ensuring critical services are healthy.

11. SENIOR ENGINEER TIP

- 💡 Design network checks based on service dependencies and business impact. Avoid simple single checks. Use multiple locations and monitoring tools to reduce false alerts and detect real issues early.

8. AUTOMATION EXAMPLE (BASH SCRIPT)

- ✓ Check DNS, port and HTTP in sequence.
- ✓ Alert if any check fails.
- ✓ Can be extended with email, Slack or monitoring tools.

```
#!/bin/bash
DOMAIN="example.com"

# 1. Check DNS
if ! dig +short $DOMAIN > /dev/null; then
    echo "ERROR: DNS resolution failed for $DOMAIN"
    exit 1
fi

# 2. Check port 80
if ! nc -z $DOMAIN 80; then
    echo "ERROR: Port 80 is not reachable on $DOMAIN"
    exit 1
fi

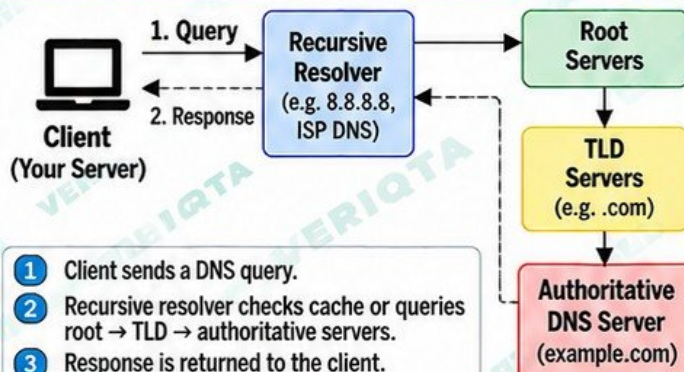
# 3. Check HTTP response
STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://$DOMAIN)
if [ "$STATUS" != "200" ]; then
    echo "ERROR: HTTP check failed with status $STATUS"
    exit 1
fi

echo "All checks passed for $DOMAIN"
```

1. WHY DETECT DNS FAILURES?

- ✓ DNS issues can break applications, even when servers are running.
- ✓ Helps identify whether the problem is local, recursive or authoritative.
- ✓ Reduces downtime and speeds up troubleshooting.
- ✓ Provides evidence for escalation to DNS or network teams.
- ✓ Improves service reliability and user experience.

2. DNS RESOLUTION FLOW



3. USING DIG FOR DNS TROUBLESHOOTING

- ✓ Query different record types (A, AAAA, MX, NS, TXT).
- ✓ Check which DNS server is responding.
- ✓ Test against specific DNS servers (recursive or authoritative).
- ✓ Identify timeouts, SERVFAIL and NXDOMAIN errors.

Examples: Using dig

```
# Query a domain (A record)
dig example.com

# Query using a specific DNS server
dig example.com @8.8.8.8

# Check MX record
dig example.com MX

# Get more detailed output
dig +trace example.com
```

4. USING resolvectl (systemd-resolved)

- ✓ View current DNS configuration.
- ✓ Check active DNS servers and link status.
- ✓ Inspect resolution statistics.
- ✓ Identify if the local resolver is the issue.

Examples: Using resolvectl

```
# Show DNS status
resolvectl status

# Show DNS servers
resolvectl dns

# Show per-link DNS information
resolvectl status eth0

# Test name resolution
resolvectl query example.com
```

5. COMMON DNS FAILURES AND HOW TO DETECT

Failure Type	Symptoms	How to Detect
No response / Timeout	Slow or no DNS response	dig example.com (check for timeout)
SERVFAIL	Server failed to resolve	dig example.com (check for SERVFAIL)
NXDOMAIN	Domain does not exist	dig example.com (check for NXDOMAIN)
Recursive resolver issue	All domains failing	Test with public DNS (e.g. 8.8.8.8)
Authoritative server issue	Specific domain failing	dig +trace example.com (check authoritative step)
Local resolver misconfiguration	Works on other servers but not this one	resolvectl status check /etc/resolv.conf

6. EVIDENCE COLLECTION

- ✓ Capture command output with timestamps.
- ✓ Test from multiple locations (local, external DNS).
- ✓ Compare results using different DNS servers.
- ✓ Collect network traces if needed (tcpdump).
- ✓ Document the exact error messages and response times.

Example: Collect DNS troubleshooting evidence

```
# Save dig output with timestamp
dig example.com +ttlid +stats > dns_check_$(date +%F_%T).log

# Capture DNS traffic (port 53)
sudo tcpdump -n -s 0 -w dns_trace.pcap port 53

# Check resolver configuration
cat /etc/resolv.conf > resolv.conf_$(date +%F_%T).txt
resolvectl status > resolvectl_status_$(date +%F_%T).txt
```

7. KEY TAKEAWAYS

- ✓ Use dig and resolvectl to quickly identify DNS issues.
- ✓ Determine if the problem is local, recursive or authoritative.
- ✓ Collect evidence before escalating.
- ✓ Test with multiple DNS servers for comparison.
- ✓ Monitor DNS health proactively to detect issues early.

8. REAL WORLD USE CASES

- ✓ Website not loading due to DNS failure.
- ✓ Application outages caused by resolver issues.
- ✓ Intermittent DNS timeouts in production.
- ✓ Misconfigured DNS after server migration.
- ✓ Detecting regional DNS problems.

9. SENIOR ENGINEER TIP

- 💡 Always test DNS resolution from multiple perspectives (local resolver, external DNS and direct authoritative server) before concluding where the problem is.

ss | connection states | SYN failures | TIME_WAIT | connection exhaustion

1. WHY MONITOR TCP CONNECTIONS?

- ✓ Detect connection issues before they impact users.
- ✓ Identify abnormal connection patterns.
- ✓ Troubleshoot application and network problems.
- ✓ Prevent connection exhaustion.
- ✓ Automate detection and alerting for faster response.

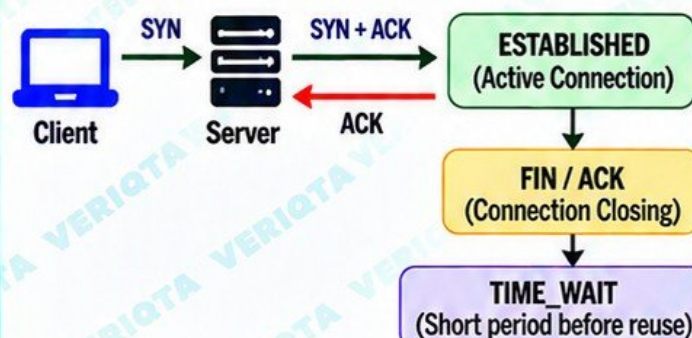
2. KEY CONCEPTS

Concept	Description
ss	Modern tool to view socket statistics (faster than netstat).
Connection states	SHOWS the current state of TCP connections (ESTABLISHED, SYN_SENT, etc).
SYN failures	Repeated failed connection attempts during the TCP handshake.
TIME_WAIT	State after a connection is closed, keeps the socket for a short period.
Connection exhaustion	Occurs when too many connections are in use, preventing new connections.

3. TCP CONNECTION STATES

State	Meaning
SYN_SENT	Client sent SYN, waiting for response.
SYN_RECV	Server received SYN, waiting for ACK.
ESTABLISHED	Connection is active.
FIN_WAIT1	Connection closing (side 1).
FIN_WAIT2	Connection closing (side 2).
TIME_WAIT	Connection closed, waiting before reuse.
CLOSE_WAIT	Remote closed, waiting for local close.
LAST_ACK	Waiting for final ACK.
CLOSED	No active connection.

4. TCP CONNECTION LIFECYCLE



5. USING ss TO VIEW CONNECTIONS

```
# Show all TCP connections with state
ss -t -a
# Show established connections
ss -t state established
# Show connections to a specific port (e.g. 80)
ss -t -a 'sport = :80'
# Show connection count by state
ss -t -a | awk '{print $1}' | sort | uniq -c | sort -nr
```

6. DETECTING SYN FAILURES

```
# Count SYN_SENT connections (possible SYN failures)
ss -t state syn-sent | wc -l
# Show top destination IPs with SYN failures
ss -t state syn-sent | awk '{print $5}' | cut -d: -f1 | \
sort | uniq -c | sort -nr | head
# Monitor in real time
watch -n 2 "ss -t state syn-sent | wc -l"
```

7. INVESTIGATING TIME_WAIT

```
# Count TIME_WAIT connections
ss -t state time-wait | wc -l
# Show top ports with TIME_WAIT
ss -t state time-wait | awk '{print $4}' | cut -d: -f2 | \
sort | uniq -c | sort -nr | head
# Check TIME_WAIT for a specific application
ss -t state time-wait '( sport = :80 )'
```

8. CONNECTION EXHAUSTION CHECKS

```
# Total number of TCP connections
ss -s
# Count by state
ss -t -a | awk '{print $1}' | sort | uniq -c | sort -nr
# Check system file descriptor limits
ulimit -n
# Check current open files lsof -n | wc -l
```

9. AUTOMATION IDEAS

- ✓ Alert when SYN_SENT count is high.
- ✓ Monitor TIME_WAIT and investigate spikes.
- ✓ Track total connections and set thresholds.
- ✓ Detect connection exhaustion early.
- ✓ Automate remediation (e.g. restart service, scale, or block offending IPs).

10. KEY TAKEAWAYS

- ✓ Use ss for faster and detailed socket information.
- ✓ Understand TCP connection states.
- ✓ Monitor SYN failures and TIME_WAIT.
- ✓ Keep an eye on total connections.
- ✓ Automate checks and alerts.
- ✓ Investigate root cause before taking action.

11. SENIOR ENGINEER TIP



A sudden increase in SYN_SENT, TIME_WAIT or total connections can indicate an application issue, DDoS attack, or resource leak. Always correlate with application logs, system metrics and network telemetry before taking action.

openssl | expiry extraction | warning thresholds | renewal verification | preventing certificate incidents

1. WHY MONITOR TLS CERTIFICATE EXPIRY?

- ✓ Prevents unexpected service outages due to expired certificates.
- ✓ Helps you plan renewals before certificates expire.
- ✓ Improves security and compliance.
- ✓ Detects misconfigured or incorrect certificates.
- ✓ Provides early alerts so you can take action in time.

2. KEY CONCEPTS

Concept	Description
TLS certificate	Digital certificate used to encrypt traffic and prove the identity of a service.
openssl	Command-line tool to view certificate information.
Expiry extraction	Method to check the certificate expiration date and remaining days.
Warning thresholds	Alert when the certificate is close to expiring (e.g., 30, 15, 7 days).
Renewal verification	Confirm that a new certificate is correctly installed and being used.
Preventing incidents	Proactive monitoring and automation to avoid expired certificate outages.

3. CHECKING CERTIFICATE EXPIRY WITH openssl

- ✓ Use openssl to view certificate details.
- ✓ Extract the expiration date.
- ✓ Calculate the number of days remaining.
- ✓ Useful for any HTTPS service (web, API, database, etc).

Example: Check certificate expiry date

```
# Check certificate expiration date
echo | openssl s_client -connect example.com:443
-servername example.com 2>/dev/null | \
openssl x509 -noout -dates
notBefore=Jun 1 00:00:00 2024 GMT
notAfter=Aug 30 23:59:59 2025 GMT
```

4. EXTRACTING AND CALCULATING DAYS REMAINING

- ✓ Extract the expiry date using openssl.
- ✓ Convert it to a readable format.
- ✓ Calculate the number of days remaining.
- ✓ Use this in scripts for automated monitoring.

Example: Get days remaining (Bash)

```
# Extract expiry date and calculate days remaining
EXPIRY_DATE=$(echo | openssl s_client -connect
example.com:443 -servername example.com 2>/dev/null | \
openssl x509 -noout -enddate | cut -d= -f2)
EXPIRY_EPOCH=$(date -d "$EXPIRY_DATE" +%s)
CURRENT_EPOCH=$(date +%s)
DAYS_REMAINING=$(( (EXPIRY_EPOCH - CURRENT_EPOCH) / 86400 ))
echo "Days remaining: $DAYS_REMAINING"
```

5. IMPLEMENTING WARNING THRESHOLDS

- ✓ Set alert thresholds (e.g. 30, 15, 7 days).
- ✓ Send notifications when a certificate is close to expiring.
- ✓ Can be integrated with monitoring tools (email, Slack, etc).
- ✓ Helps avoid last minute incidents.

Example: Alert if certificate is expiring soon

```
# Alert if certificate expires in less than 30 days
THRESHOLD=30
if [ "$DAYS_REMAINING" -le "$THRESHOLD" ]; then
echo "WARNING: Certificate for example.com \
"expires in $DAYS_REMAINING days!"
# Add your alert command here (email, Slack, etc.)
fi
```

6. RENEWAL VERIFICATION

- ✓ After renewal, verify the new certificate is installed.
- ✓ Check the new expiry date.
- ✓ Ensure the correct certificate is being used (especially if multiple certificates exist).
- ✓ Test the service to confirm there are no issues.

Example: Verify new certificate after renewal

```
# Check new certificate expiry date
echo | openssl s_client -connect example.com:443 \
-servername example.com 2>/dev/null | \
openssl x509 -noout -dates

# Test the HTTPS service
curl -I https://example.com | head -n 5
```

7. KEY TAKEAWAYS

- ✓ Monitor certificate expiry proactively.
- ✓ Use openssl to extract and check expiry dates.
- ✓ Set warning thresholds and send alerts.
- ✓ Verify certificates after renewal.
- ✓ Automate checks to prevent certificate incidents.

8. REAL WORLD USE CASES

- ✓ Web servers (Nginx, Apache, IIS).
- ✓ APIs and microservices.
- ✓ Databases with TLS (PostgreSQL, MySQL).
- ✓ Load balancers and reverse proxies.
- ✓ Internal and external services.
- ✓ Wildcard and multiple domain certificates.

9. SENIOR ENGINEER TIP

- 💡 Monitor all certificates, including internal and wildcard certificates. Automate renewal where possible (e.g. with Let's Encrypt or your PKI). Always verify the new certificate is correctly installed and serving traffic before closing the change.

Firewall Automation Without Locking Yourself Out

nftables | validation | staged application | SSH protection | rollback strategy

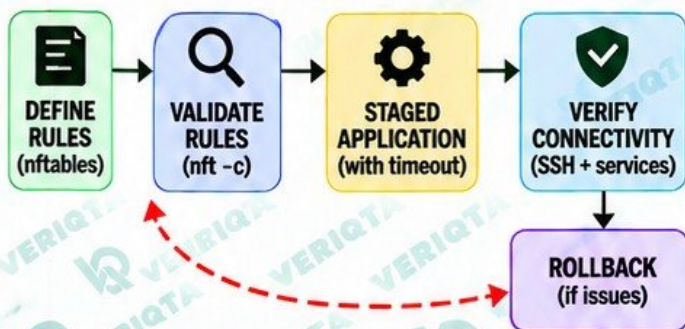
1. WHY FIREWALL AUTOMATION?

- ✓ Enforce consistent and secure network policies across multiple servers.
- ✓ Reduce manual errors and configuration drift.
- ✓ Protect critical services such as SSH.
- ✓ Validate rules before applying to avoid lockouts.
- ✓ Use staged application and rollback to stay safe.

2. KEY CONCEPTS

Concept	Description
nftables	Modern Linux firewall framework (replaces iptables).
Validation	Check rules for syntax and logic errors before applying.
Staged application	Test rules in a controlled way (e.g. temporary apply with timeout).
SSH protection	Always allow existing SSH connections and keep a safe management path.
Rollback strategy	Automatically revert to previous rules if connectivity is lost.

3. FIREWALL AUTOMATION FLOW



4. EXAMPLE: BASIC nftables RULES

```
# /etc/nftables.conf (basic example)
table inet filter {
  chain input {
    type filter hook input priority 0;
    policy drop;

    # Allow established connections
    ct state established,related accept

    # Allow SSH (change port if needed)
    tcp dport 22 accept

    # Allow loopback
    iif "lo" accept
  }
}
```

5. VALIDATING RULES BEFORE APPLYING

```
# Check syntax and load test (no change applied)
sudo nft -c -f /etc/nftables.conf

# List current rules
sudo nft list ruleset
```

6. STAGED APPLICATION WITH TIMEOUT

```
# Apply rules with automatic rollback after 60 seconds
sudo nft -f /etc/nftables.conf
sudo sleep 60
# If still connected, make the changes permanent
sudo systemctl restart nftables
# If disconnected, rules will revert automatically
# (use a separate session to confirm)
```

7. SSH PROTECTION BEST PRACTICES

- ✓ Always allow existing SSH connections.
- ✓ Test rules from a separate session before closing the first session.
- ✓ Consider allowing SSH from specific IP ranges only.
- ✓ Use key-based authentication (no passwords).
- ✓ Keep a fallback access method (e.g. console or cloud provider access).

8. ROLLBACK STRATEGY

- ✓ Keep a backup of the previous rules.
- ✓ Use timed rollback when applying new rules.
- ✓ Have a script to quickly restore last known good rules.
- ✓ Monitor connectivity after applying.
- ✓ Document and log all changes.

9. EXAMPLE: ROLLBACK SCRIPT

```
#!/bin/bash
# Restore previous firewall rules
sudo cp /etc/nftables.conf.bak /etc/nftables.conf
sudo systemctl restart nftables
echo "Firewall rules rolled back."
```

10. KEY TAKEAWAYS

- ✓ Validate before applying.
- ✓ Use staged rollout with timeout.
- ✓ Always protect SSH access.
- ✓ Have a rollback plan.
- ✓ Test changes and monitor connectivity.

11. SENIOR ENGINEER TIP



Automate firewall changes with safety checks, timeouts and automatic rollback. Never apply untested rules directly to production servers.

SSH keys | host verification | BatchMode | timeouts | bounded parallelism | partial fleet failures

1. WHY AUTOMATE SSH SAFELY?

- ✓ Allows you to run commands and automate tasks across many Linux servers.
- ✓ SSH keys enable secure and passwordless access.
- ✓ Host verification prevents man-in-the-middle attacks.
- ✓ Using timeouts and bounded parallelism avoids hanging or overloading your systems.
- ✓ Handles partial fleet failures so one failed host does not stop the entire process.

2. KEY CONCEPTS

Concept	Description
SSH keys	Use key-based authentication instead of passwords.
Host verification	Verify server identity using <code>known_hosts</code> to prevent spoofing.
BatchMode	Disables interactive password prompts (suitable for automation).
Timeouts	Set connection and command timeouts to avoid hanging.
Bounded parallelism	Limit the number of concurrent SSH connections.
Partial fleet failures	Continue automation even if some servers fail.

3. SETTING UP SSH KEYS

- ✓ Generate an SSH key pair if you don't have one.
- ✓ Copy the public key to all target servers.
- ✓ Use a dedicated key for automation.
- ✓ Set proper permissions on keys.

Example: Generate and copy SSH key

```
# Generate SSH key pair
ssh-keygen -t ed25519 -C "automation@veriqta"
# Copy public key to a server
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server1
# Set key permissions
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_ed25519
```

4. HOST VERIFICATION

- ✓ Use `known_hosts` to store server fingerprints.
- ✓ Verify host keys before connecting.
- ✓ Avoid disabling host key checking in production.
- ✓ Manage multiple hosts safely (`ssh-keyscan` or manual verification).

Example: Verify and add host keys

```
# View a server's host key
ssh-keyscan -H server1.example.com
# Add to known_hosts (manual verification)
ssh-keyscan -H server1.example.com >> ~/.ssh/known_hosts
# Check existing host key
ssh-keygen -F server1.example.com
```

5. USING BatchMode AND TIMEOUTS

- ✓ BatchMode prevents password prompts.
- ✓ Set connection and command timeouts.
- ✓ Fail fast if a server is unreachable.
- ✓ Useful for non-interactive automation scripts.

Example: SSH with BatchMode and timeouts

```
ssh -o BatchMode=yes \
-o ConnectTimeout=10 \
-o ServerAliveInterval=5 \
-o ServerAliveCountMax=2 \
user@server1.example.com "uptime"
```

6. BOUNDED PARALLELISM

- ✓ Limit the number of concurrent SSH connections.
- ✓ Avoid overwhelming servers or your network.
- ✓ Use tools like `xargs`, `GNU parallel` or a simple bash loop.
- ✓ Adjust the concurrency level based on your environment.

Example: Run a command on multiple servers

```
# Using xargs with a limit of 5 concurrent connections
cat servers.txt | xargs -n 1 -P 5 -I {} \
ssh -o BatchMode=yes -o ConnectTimeout=10 \
{} "hostname && uptime"
```

7. HANDLING PARTIAL FLEET FAILURES

- ✓ Continue execution even if some servers fail.
- ✓ Log successes and failures separately.
- ✓ Use timeouts to prevent hanging.
- ✓ Review failed hosts and retry if needed.

Example: Handle failures and log results

```
cat servers.txt | while read host; do
if ssh -o BatchMode=yes -o ConnectTimeout=10 \
$host "uptime" >> success.log 2>> error.log; then
echo "$host SUCCESS"
else
fi echo "$host FAILED"
done
```

8. REAL WORLD USE CASES

- ✓ Run system updates across many servers.
- ✓ Collect logs or system information.
- ✓ Deploy configuration changes.
- ✓ Monitor server health.
- ✓ Execute automated maintenance tasks.
- ✓ Handle environments with hundreds of servers safely.

10. KEY TAKEAWAYS

- ✓ Use SSH keys and verify host identity.
- ✓ Always use BatchMode and timeouts.
- ✓ Limit parallel connections.
- ✓ Handle partial failures gracefully.
- ✓ Log and review results.
- ✓ Plan for security and reliability.

9. SENIOR ENGINEER TIP



- ✓ Use a bastion host or VPN for access to private servers.
- ✓ Keep separate keys for automation and personal use.
- ✓ Monitor and alert on repeated SSH failures.
- ✓ Implement retry logic with backoff for transient issues.
- ✓ Regularly rotate SSH keys.

xargs -P | GNU Parallel concepts | concurrency limits | rate limiting | blast-radius control

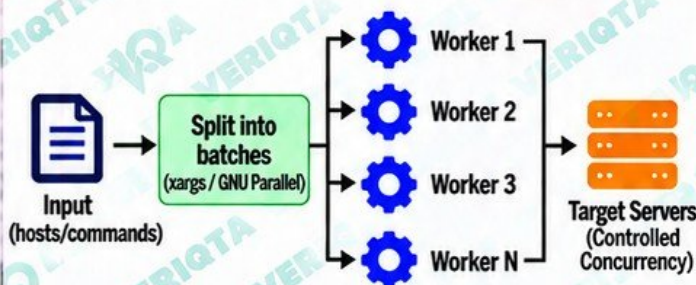
1. WHY PARALLEL AUTOMATION?

- ✓ Run commands on many hosts or items at the same time.
- ✓ Reduce execution time for large tasks.
- ✓ Control concurrency to avoid overloading systems.
- ✓ Limit the impact of failures (blast-radius control).
- ✓ Use rate limiting and batching for safer automation.

2. KEY CONCEPTS

Concept	Description
xargs -P	Runs multiple commands in parallel with a specified number of processes.
GNU Parallel	Advanced tool for parallel execution with better control and features.
Concurrency limits	Limit the number of simultaneous commands or connections.
Rate limiting	Control the rate of requests to avoid overloading systems (e.g. sleep, --delay).
Blast-radius control	Restrict scope (batches, groups, limits) to prevent large-scale impact.

3. HOW PARALLEL AUTOMATION WORKS



4. USING xargs -P (BASIC EXAMPLE)

```
# Run a command on multiple hosts with 5 parallel processes
cat hosts.txt | xargs -P 5 -I {} ssh {} 'uptime'

# Copy a file to multiple hosts in parallel
cat hosts.txt | xargs -P 10 -I {} scp myfile.txt {}:~/tmp/

# Run a local command on multiple items
ls *.log | xargs -P 4 -I {} gzip {}
```

5. USING GNU PARALLEL

```
# Install (if not installed)
sudo apt install parallel

# Run a command on multiple hosts with concurrency limit
cat hosts.txt | parallel -j 5 ssh {} 'uptime'

# Copy a file with progress and nicer output
cat hosts.txt | parallel -j 10 scp myfile.txt {}:~/tmp/

# Run a command with delay (rate limiting)
cat hosts.txt | parallel -j 5 --delay 2 ssh {} 'systemctl status nginx'
```

6. CONCURRENCY LIMITS AND RATE LIMITING

```
# Limit to 5 concurrent processes
cat hosts.txt | xargs -P 5 -I {} ssh {} 'hostname'

# GNU Parallel with job limit
cat hosts.txt | parallel -j 5 ssh {} 'hostname'

# Add delay between jobs (rate limiting)
cat hosts.txt | parallel -j 5 --delay 3 ssh {} 'uptime'

# Limit based on load (GNU Parallel)
cat hosts.txt | parallel -j 5 --load 70% ssh {} 'uptime'
```

7. BLAST-RADIUS CONTROL STRATEGIES

- ✓ Use batches (e.g. process 10-20 hosts at a time).
- ✓ Set conservative concurrency limits.
- ✓ Use rate limiting (delays between requests).
- ✓ Target specific groups instead of the entire fleet.
- ✓ Monitor system health during execution.
- ✓ Have a rollback or stop mechanism if issues are detected.

8. REAL-WORLD EXAMPLE: ROLLING UPDATES

```
# Update a package on 20 hosts with 5 concurrent connections
cat hosts.txt | head -20 | parallel -j 5 'ssh {} \
  sudo apt update && sudo apt upgrade -y'

# Check service health after update
cat hosts.txt | head -20 | parallel -j 5 'ssh {} \
  systemctl is-active nginx'
```

9. KEY TAKEAWAYS

- ✓ Parallel automation saves time but must be controlled.
- ✓ Use xargs -P or GNU Parallel for concurrency.
- ✓ Set appropriate limits and use rate limiting.
- ✓ Process in batches to reduce risk.
- ✓ Monitor and log results.
- ✓ Always have a plan to stop or roll back if needed.

10. COMMON USE CASES

- ✓ Running health checks across many servers.
- ✓ Deploying configuration changes.
- ✓ Collecting logs or metrics.
- ✓ Restarting services in batches.
- ✓ Scanning for vulnerabilities.
- ✓ Applying security patches at scale.

11. SENIOR ENGINEER TIP



Parallel execution can multiply both success and failure. Always start small, use conservative limits, monitor closely and automate safety checks to prevent a fleet-wide outage.

33 Ansible: From Scripts to Desired State

inventory | modules | playbooks | handlers | idempotency

1. WHY USE ANSIBLE?

- ✓ Move from manual scripts to a consistent desired state.
- ✓ Automate configuration, deployment and operations.
- ✓ Simple and human-readable (YAML).
- ✓ Agentless (uses SSH).
- ✓ Idempotent (safe to run multiple times).
- ✓ Supports large fleets and complex environments.

2. KEY CONCEPTS

Concept	Description
Inventory	List of managed hosts (static or dynamic).
Modules	Pre-built units to perform tasks (e.g. package, service, copy, user).
Playbooks	YAML files that define the desired state for your infrastructure.
Handlers	Tasks that run only when notified (e.g. restart a service).
Idempotency	Running the same playbook multiple times results in the same state without changes if already configured.

3. ANSIBLE INVENTORY

- ✓ Define your hosts in a static inventory file.
- ✓ Use groups to organize servers.
- ✓ You can also use dynamic inventory (e.g. from cloud).
- ✓ Store inventory in version control.

Example: Static inventory (inventory.ini)

```
# inventory.ini
[webservers]
web1 ansible_host=192.168.1.10
web2 ansible_host=192.168.1.11

[dbservers]
db1 ansible_host=192.168.1.20
db2 ansible_host=192.168.1.21
```

4. ANSIBLE MODULES

- ✓ Use modules to perform common tasks.
- ✓ No need to write complex scripts.
- ✓ Modules are idempotent and easy to use.
- ✓ Examples: package, service, copy, user, file, command.

Example: Using modules

```
# Install a package
ansible webservers -m package -a "name=nginx state=present"

# Start and enable a service
ansible webservers -m service -a "name=nginx state=started enabled=yes"

# Create a user
ansible all -m user -a "name=devops state=present shell=/bin/bash"
```

5. ANSIBLE PLAYBOOKS

- ✓ Playbooks define the desired state in YAML.
- ✓ Contain multiple tasks executed in order.
- ✓ Can target specific hosts or groups.
- ✓ Easy to read, version and maintain.

Example: Simple playbook (site.yml)

```
---
- name: Configure web servers
  hosts: webservers
  become: yes
  tasks:
    - name: Install nginx
      ansible.builtin.package:
        name: nginx
        state: present
    - name: Start nginx service
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: yes
```

6. ANSIBLE HANDLERS

- ✓ Handlers run only when notified by a task.
- ✓ Used for actions like restarting a service.
- ✓ Prevent unnecessary changes and improve efficiency.

Example: Playbook with handler

```
---
- name: Install and configure nginx
  hosts: webservers
  become: yes
  tasks:
    - name: Copy nginx configuration
      ansible.builtin.copy:
        src: nginx.conf
        dest: /etc/nginx/nginx.conf
        notify: Restart nginx
  handlers:
    - name: Restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

7. IDEMPOTENCY

- ✓ Ansible ensures the desired state is achieved.
- ✓ Running the same playbook multiple times makes no unintended changes.
- ✓ Modules check the current state before applying changes.
- ✓ Keeps your infrastructure consistent and safe.

8. REAL WORLD USE CASES

- ✓ Configure and maintain web servers.
- ✓ Deploy applications.
- ✓ Manage users, packages and services.
- ✓ Enforce security configurations.
- ✓ Automate multi-environment deployments.
- ✓ Manage cloud and on-premise infrastructure.

9. SENIOR ENGINEER TIP

- 💡 Use Ansible roles, variables and environment-specific inventories (dev, staging, prod). Always test playbooks in a non-production environment before running in production.

roles | variables | secrets | check mode | serial rollout | failure handling

1. WHY PRODUCTION-GRADE PLAYBOOKS?

- ✓ Automate infrastructure and application configuration at scale.
- ✓ Ensure consistency, repeatability and auditability.
- ✓ Use roles, variables and secrets for maintainable design.
- ✓ Validate changes safely with check mode.
- ✓ Roll out changes in a controlled way.
- ✓ Handle failures gracefully and keep your systems stable.

2. KEY CONCEPTS

Concept	Description
Roles	Reusable structure for organizing tasks, handlers, files and templates.
Variables	Externalize configuration for flexibility (e.g. <code>group_vars</code> , <code>host_vars</code>).
Secrets	Protect sensitive data using Ansible Vault or external secret stores.
Check mode	Simulate changes without making any modifications.
Serial rollout	Deploy to a limited number of hosts at a time to reduce risk.
Failure handling	Use strategies like <code>ignore_errors</code> , <code>rescue</code> and retries to handle failures gracefully.

3. USING ROLES

- ✓ Organize playbooks into reusable roles.
- ✓ Keep a clean directory structure.
- ✓ Make roles configurable with variables.
- ✓ Use roles for different environments (dev, staging, prod).

Example: Role structure

```
roles/
├── nginx/
│   ├── tasks/
│   ├── handlers/
│   ├── templates/
│   ├── files/
│   ├── vars/
│   └── defaults/
```

4. VARIABLES AND INVENTORY

- ✓ Use inventory to define your hosts.
- ✓ Use `group_vars` and `host_vars` for environment-specific values.
- ✓ Keep common values in defaults.
- ✓ Override values for different environments.

Example: `group_vars` (production.yml)

```
---
nginx_port: 80
nginx_workers: 4
environment: production
db_host: db.example.com
db_user: appuser
```

5. MANAGING SECRETS

- ✓ Use Ansible Vault to encrypt sensitive data.
- ✓ Never hardcode passwords, API keys or tokens.
- ✓ Keep vaulted files in version control.
- ✓ Use external secret stores if required (e.g. HashiCorp Vault).

Example: Encrypted variables file

```
# encrypt a variables file
ansible-vault encrypt group_vars/production.yml

# use in a playbook
- hosts: all
  vars_files:
    - group_vars/production.yml
```

6. CHECK MODE (DRY RUN)

- ✓ Preview changes without making modifications.
- ✓ Validate playbooks and catch issues early.
- ✓ Useful in CI/CD pipelines and during maintenance windows.

Example: Run playbook in check mode

```
# Run playbook without making changes
ansible-playbook site.yml --check

# See what would change
TASK [nginx : Install nginx] *****
changed: [web1] (DRY RUN)
```

7. SERIAL ROLLOUT

- ✓ Deploy to a limited number of hosts at a time.
- ✓ Reduces risk and avoids full outage.
- ✓ Useful for large fleets and critical services.

Example: Playbook with serial rollout

```
- hosts: webserver
  serial: 2
  roles:
    - nginx
```

8. FAILURE HANDLING

- ✓ Use `ignore_errors` for non-critical tasks.
- ✓ Use `rescue` to recover from failures.
- ✓ Set retries and delay for unstable tasks.
- ✓ Collect logs and notify when failures occur.

Example: Task with failure handling

```
- name: Restart nginx with retry
  ansible.builtin.service:
    name: nginx
    state: restarted
  register: result
  retries: 3
  delay: 10
  until: result is succeeded
```

9. KEY TAKEAWAYS

- ✓ Use roles for scalable and maintainable playbooks.
- ✓ Externalize configuration with variables.
- ✓ Protect secrets using Ansible Vault or external tools.
- ✓ Always validate changes with check mode.
- ✓ Use serial rollout for safer deployments.
- ✓ Implement proper failure handling to keep your systems stable.

10. SENIOR ENGINEER TIP

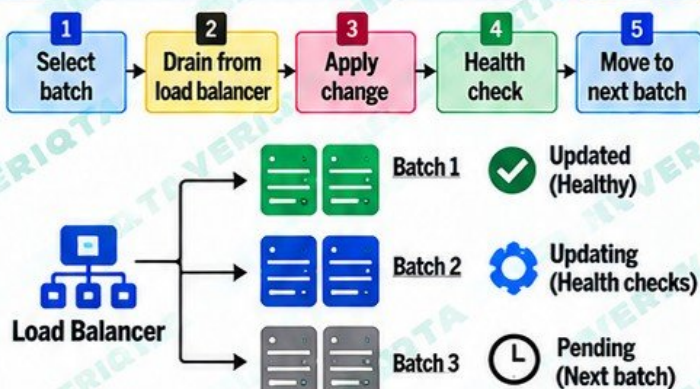
- 💡 Integrate playbooks with version control, CI/CD and monitoring for fully automated and auditable deployments.

batches | health gates | load balancer draining | canaries | rollback | preventing fleet-wide outages

1. WHY USE ROLLING CHANGES?

- ✓ Reduce risk by updating a small group of servers at a time.
- ✓ Maintain service availability during changes.
- ✓ Detect issues early using health gates and canaries.
- ✓ Allow quick rollback if something goes wrong.
- ✓ Prevent fleet-wide outages.
- ✓ Provide a controlled and repeatable deployment process.

2. ROLLING CHANGE FLOW



3. DEPLOYING IN BATCHES

- ✓ Split your servers into small batches (e.g. 10% or 5 servers).
- ✓ Update one batch at a time.
- ✓ Verify health before moving to the next batch.
- ✓ Limit the blast radius if something goes wrong.

Example: Ansible serial rollout

```

- name: Rolling update with batches
  hosts: webservers
  serial: 10% # or a fixed number like 5
  tasks:
    - name: Update application
      ansible.builtin.package:
        name: myapp
        state: latest
  
```

4. HEALTH GATES

- ✓ Check application health after each batch.
- ✓ Verify key metrics (HTTP 200, response time, error rate).
- ✓ Stop the rollout if health checks fail.
- ✓ Use automated checks instead of manual verification.

Example: Health check script

```

#!/bin/bash
URL="http://localhost/health"
HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" $URL)
if [ "$HTTP_CODE" = "200" ]; then
  echo "Healthy"
  exit 0
else
  echo "Health check failed (HTTP $HTTP_CODE)"
  exit 1
  
```

5. LOAD BALANCER DRAINING

- ✓ Remove servers from the load balancer before updating.
- ✓ Allow existing connections to complete.
- ✓ Add the server back only after it passes health checks.
- ✓ Prevents user impact during the update.

Example: Nginx drain and add back

```

# Mark server as down (drain)
curl -X POST http://lb-api/servers/web1/drain
# After update and health check
curl -X POST http://lb-api/servers/web1/enable
  
```

6. CANARY DEPLOYMENTS

- ✓ Update a small number of servers first (canary).
- ✓ Monitor closely for errors or performance issues.
- ✓ Gradually increase the rollout if the canary is healthy.
- ✓ Stop and rollback immediately if issues are detected.

Example: Canary rollout (concept)



7. ROLLBACK STRATEGY

- ✓ Have a tested rollback plan before starting.
- ✓ Keep previous version available.
- ✓ Automate rollback if health checks fail.
- ✓ Verify the rollback and restore the server to the load balancer.

Example: Rollback using Ansible

```

- name: Rollback to previous version
  hosts: webservers
  tasks:
    - name: Install previous version
      ansible.builtin.package:
        name: myapp
        state: 1.2.3 # specify previous version
  
```

8. PREVENTING FLEET-WIDE OUTAGES

- ✓ Use small batches and canaries.
- ✓ Always use health gates.
- ✓ Monitor key metrics during the rollout.
- ✓ Automate rollback on failure.
- ✓ Keep changes reversible.
- ✓ Test the process in a staging environment first.

9. KEY TAKEAWAYS

- ✓ Rolling changes reduce risk and maintain availability.
- ✓ Batches, health gates and canaries detect issues early.
- ✓ Drain servers from the load balancer before updating.
- ✓ Have a solid rollback strategy.
- ✓ Automate and monitor everything to prevent fleet-wide outages.

10. SENIOR ENGINEER TIP

- 💡 Combine load balancer draining with automated health checks and rollback.
- ✓ Monitor application metrics (error rate, latency, traffic).
- ✓ Use canary deployments for higher-risk changes.
- ✓ Always test the entire process in a non-production environment.
- ✓ Document the rollout and rollback procedures.

1. WHY AUTOMATE LINUX SECURITY BASELINES?

- ✓ Enforces consistent security settings across all servers.
- ✓ Reduces human error and configuration drift.
- ✓ Improves detection of security risks early.
- ✓ Provides audit-ready evidence.
- ✓ Helps you meet compliance requirements.
- ✓ Avoids manual effort and keeps systems secure at scale.

2. KEY CONCEPTS

Concept	Description
Permissions	Correct file and directory permissions to protect sensitive data.
SSH configuration	Secure SSH settings to prevent unauthorized access.
Services	Manage and disable unnecessary services.
Audit controls	Logging, auditing and monitoring for security events.
Validation	Check current state and verify compliance after changes.
Avoiding destructive remediation	Make safe, targeted changes instead of blanket rules that can break systems.

3. AUTOMATING FILE AND DIRECTORY PERMISSIONS

- ✓ Identify sensitive files and directories.
- ✓ Set correct ownership and permissions.
- ✓ Detect and fix insecure permissions.
- ✓ Exclude system files that should not be changed.

Example: Fix permissions on critical directories

```
# Set secure permissions on /etc and /var
sudo chmod 755 /etc
sudo chmod 755 /var
sudo find /etc -type f -exec chmod 644 {} \;
sudo find /etc -type d -exec chmod 755 {} \;
```

4. SECURING SSH CONFIGURATION

- ✓ Disable root login.
- ✓ Use key-based authentication.
- ✓ Set secure settings (e.g. timeout, max attempts).
- ✓ Validate configuration before applying.

Example: Secure SSH configuration (sshd_config)

```
# Key security settings
PermitRootLogin no
PasswordAuthentication no
MaxAuthTries 3
ClientAliveInterval 300
ClientAliveCountMax 2
```

5. MANAGING SERVICES

- ✓ Identify unnecessary services.
- ✓ Disable and stop services not needed.
- ✓ Ensure critical services are enabled and running.
- ✓ Validate service state after changes.

Example: Disable and stop an unnecessary service

```
# Disable and stop a service (e.g. telnet)
sudo systemctl disable telnet
sudo systemctl stop telnet
# Verify status
systemctl status telnet
```

6. AUDIT CONTROLS AND LOGGING

- ✓ Enable and configure audit logging.
- ✓ Monitor key files, commands and configuration changes.
- ✓ Send logs to a central system if possible.
- ✓ Regularly review audit logs.

Example: Add audit rule for /etc/passwd

```
# Monitor changes to /etc/passwd
sudo auditctl -w /etc/passwd -p wa -k passwd_changes
# List audit rules
sudo auditctl -l
```

7. VALIDATION AND COMPLIANCE CHECKS

- ✓ Run validation checks after applying changes.
- ✓ Compare current state with security baselines.
- ✓ Use tools (e.g. Ansible, OpenSCAP) for automated checks.
- ✓ Generate reports for audit and compliance.

Example: Check SSH root login is disabled

```
# Validate SSH root login
grep -E '^PermitRootLogin' /etc/ssh/sshd_config
```

8. AVOIDING DESTRUCTIVE REMEDIATION

- ✓ Do not apply blanket changes without testing.
- ✓ Exclude critical systems and files.
- ✓ Use phased rollouts (test first, then production).
- ✓ Keep backups or rollback plans.
- ✓ Make targeted, minimal changes.

Example: Safe check before change

```
# Check current permission before changing
ls -ld /etc/ssh
# Only change if needed
if [ "$(stat -c %a /etc/ssh)" != "755" ]; then
  sudo chmod 755 /etc/ssh
fi
```

9. KEY TAKEAWAYS

- ✓ Automate security baselines for consistency and compliance.
- ✓ Secure permissions, SSH, services and audit controls.
- ✓ Always validate changes before applying.
- ✓ Avoid blanket remediation that can break systems.
- ✓ Use phased rollouts and rollback plans.
- ✓ Keep monitoring and regularly review logs.

10. SENIOR ENGINEER TIP

- 💡 Integrate security baseline automation into your CI/CD pipeline and configuration management for continuous compliance.

environment variables | secret stores | file permissions | process exposure | rotation | preventing credentials from entering logs

1. WHY SECURE SECRETS IN AUTOMATION?

- ✓ Protects sensitive credentials from exposure.
- ✓ Reduces risk of unauthorized access.
- ✓ Prevents accidental leaks in logs, process lists and code repositories.
- ✓ Supports compliance and security best practices.
- ✓ Ensures safe rotation and access control.

2. KEY CONCEPTS

Concept	Description
Environment variables	Store secrets in environment variables (in memory, not in code).
Secret stores	Use tools like HashiCorp Vault, AWS Secrets Manager or cloud key management services.
File permissions	Restrict access to sensitive files using correct ownership and permissions.
Process exposure	Avoid exposing secrets in process lists (e.g. ps, top).
Rotation	Regularly rotate credentials to reduce risk.
Preventing credentials from entering logs	Ensure secrets are not printed in logs, command output or error messages.

3. USING ENVIRONMENT VARIABLES

- ✓ Store secrets in environment variables.
- ✓ Avoid hardcoding credentials in scripts.
- ✓ Use temporary variables for automation.
- ✓ Clear sensitive variables after use.

Example: Set and use environment variables

```
# Set a secret (temporary for current session)
export DB_PASSWORD="StrongPass123!"
# Use the variable in a command
mysql -u dbuser -p"$DB_PASSWORD"
# Clear the variable after use
unset DB_PASSWORD
```

4. USING SECRET STORES

- ✓ Use a dedicated secret store for production.
- ✓ Control access with proper authentication.
- ✓ Avoid storing secrets in plain text.
- ✓ Integrate with automation tools (e.g. Ansible, scripts).

Example: Retrieve a secret from HashiCorp Vault

```
# Login to Vault (using token)
export VAULT_ADDR="https://vault.example.com"
export VAULT_TOKEN="your-token"

# Retrieve a secret
vault kv get -field=password secret/db
```

5. FILE PERMISSIONS

- ✓ Store secrets in restricted files.
- ✓ Use correct ownership and permissions.
- ✓ Avoid world-readable files.
- ✓ Verify permissions regularly.

Example: Secure a secret file

```
# Create a secret file
echo "API_KEY=abc123" > /etc/myapp/secret.env
# Restrict permissions
chmod 600 /etc/myapp/secret.env
chown root:root /etc/myapp/secret.env
```

6. AVOIDING PROCESS EXPOSURE

- ✓ Do not pass secrets directly on the command line.
- ✓ Use environment variables or secure files instead.
- ✓ Be aware that process arguments can be visible using ps or top.
- ✓ Use tools that read secrets from files or secret stores.

Example: Avoid passing secrets on command line

```
# Insecure (visible in process list)
psql -U dbuser -W -h db.example.com -p 5432

# Secure (use environment variable or .pgpass file)
PGPASSWORD="$DB_PASSWORD" psql -U dbuser -h db.example.com
```

7. ROTATION OF CREDENTIALS

- ✓ Rotate credentials regularly.
- ✓ Automate rotation where possible.
- ✓ Update dependent services after rotation.
- ✓ Remove old credentials securely.

Example: Rotate a database password

```
# Generate a new password (example)
NEW_PASS=$(openssl rand -base64 32)
# Update in secret store or database
echo "Rotating password..."
# Apply the new password (example command)
mysql -u admin -p -e "ALTER USER 'app'@'%' IDENTIFIED BY '$NEW_PASS';"
```

8. PREVENTING CREDENTIALS IN LOGS

- ✓ Do not print secrets in logs.
- ✓ Mask sensitive values in output.
- ✓ Use logging libraries or tools that support redaction.
- ✓ Avoid verbose debugging in production.

Example: Mask secrets in logs (Bash)

```
# Function to mask a secret value
mask() {
  local value="$1"
  echo "${value:0:2}*****${value: -2}"
}

echo "Connecting with password: $(mask $DB_PASSWORD)"
```

9. KEY TAKEAWAYS

- ✓ Use environment variables or secret stores.
- ✓ Set correct file permissions.
- ✓ Avoid exposing secrets in process lists.
- ✓ Rotate credentials regularly.
- ✓ Prevent credentials from entering logs.
- ✓ Follow the principle of least privilege.

10. SENIOR ENGINEER TIP



Integrate secret management with your automation pipeline (e.g. Ansible + Vault) and enforce log redaction to prevent accidental exposure.

Logs | metrics | exit status | duration | failure rate | last successful execution | alerting on outcomes

1. WHY OBSERVABLE AUTOMATION?

- ✓ Know what ran, what changed and what failed.
- ✓ Detect issues early before they impact systems.
- ✓ Measure reliability and performance over time.
- ✓ Improve troubleshooting with logs and metrics.
- ✓ Alert on outcomes, not just execution.
- ✓ Build trust and confidence in automation.

2. KEY CONCEPTS

Concept	Description
Logs	Record what happened during execution (e.g. start, changes, errors).
Metrics	Quantify execution (e.g. success, failure, duration, resources).
Exit status	Check return code to determine success or failure (0 = success).
Duration	Measure how long the automation took.
Failure rate	Track percentage of failed runs over time.
Last successful execution	Know when it last ran successfully.
Alerting on outcomes	Send alerts based on success, failure or unexpected changes (not just when it runs).

3. LOGGING AUTOMATION

- ✓ Log key events (start, end, changes, errors).
- ✓ Include useful context (host, timestamp, user).
- ✓ Use structured logs (JSON) for easy analysis.
- ✓ Store logs in a central location.

Example: Log automation with timestamps

```
#!/bin/bash
LOG_FILE="/var/log/automation.log"
echo "$(date '+%Y-%m-%d %H:%M:%S') | START | \
Update packages" >> $LOG_FILE
apt update && apt -y upgrade
EXIT_CODE=$?
echo "$(date '+%Y-%m-%d %H:%M:%S') | END | \
Exit code: $EXIT_CODE" >> $LOG_FILE
```

4. COLLECTING AND EXPOSING METRICS

- ✓ Track success and failure counts.
- ✓ Measure execution duration.
- ✓ Export metrics to monitoring tools (e.g. Prometheus).
- ✓ Monitor trends over time.

Example: Simple metrics (Prometheus format)

```
#!/bin/bash
METRICS_FILE="/var/lib/node_exporter/textfile_collector/
automation.prom"
echo "automation_success_total 1" > $METRICS_FILE
echo "automation_failure_total 0" >> $METRICS_FILE
echo "automation_duration_seconds 42" >> $METRICS_FILE
echo "automation_last_success_timestamp $(date +%s)" >>
$METRICS_FILE
```

5. CHECKING EXIT STATUS

- ✓ Use exit codes to detect success or failure.
- ✓ Capture and handle errors properly.
- ✓ Log meaningful error messages.
- ✓ Trigger alerts if the exit code is non-zero.

Example: Handle exit status

```
#!/bin/bash
run_backup.sh
if [ $? -eq 0 ]: then
    echo "Backup completed successfully"
else
    echo "Backup failed! Check logs."
    exit 1
fi
```

6. TRACKING DURATION

- ✓ Measure how long automation tasks take.
- ✓ Identify slow or stuck executions.
- ✓ Alert if duration exceeds expected time.
- ✓ Use duration data to optimize performance.

Example: Measure execution time

```
#!/bin/bash
START_TIME=$(date +%s)
# your automation command here
sleep 10
END_TIME=$(date +%s)
DURATION=$((END_TIME - START_TIME))
echo "Execution time: ${DURATION} seconds"
```

7. MONITORING FAILURE RATE

- ✓ Track total runs vs failed runs.
- ✓ Calculate failure rate over time.
- ✓ Set thresholds for acceptable failure rate.
- ✓ Investigate and fix recurring failures.

Example: Calculate failure rate (simple)

```
#!/bin/bash
TOTAL=100
FAILED=5
RATE=$(( FAILED * 100 / TOTAL ))
echo "Failure rate: ${RATE}%"
```

8. ALERTING ON OUTCOMES

- ✓ Alert on failures, high failure rate or unexpected changes.
- ✓ Use email, Slack or monitoring tools.
- ✓ Include useful context in alerts.
- ✓ Focus on the result, not just that it ran.

Example: Send alert on failure

```
#!/bin/bash
if [ $? -ne 0 ]: then
    echo "Automation failed on $(hostname)" | \
    mail -s "Automation Alert" ops@example.com
fi
```

9. KEY TAKEAWAYS

- ✓ Collect and centralize logs.
- ✓ Track metrics, duration and failure rate.
- ✓ Monitor last successful execution.
- ✓ Alert on outcomes, not just execution.
- ✓ Use this data to improve reliability and prevent incidents.

10. SENIOR ENGINEER TIP

- 💡 Integrate your automation metrics and logs with your monitoring stack (e.g. Prometheus + Grafana or ELK) for real-time visibility and proactive alerting.

39 Incident-Safe Automation and Rollback

Stop conditions | kill switches | checkpoints | state capture | rollback | recovery after partial execution | preserving forensic evidence

1. WHY INCIDENT-SAFE AUTOMATION?

- ✓ Reduces the impact of mistakes and unexpected failures.
- ✓ Provides a safe way to stop and rollback changes.
- ✓ Helps recovery after partial execution.
- ✓ Preserves evidence for investigation.
- ✓ Increases confidence when making changes in production.

2. KEY CONCEPTS

Concept	Description
Stop conditions	Define rules to stop the automation when something is wrong (e.g. error rate, health check).
Kill switches	A manual or automated way to immediately stop the process.
Checkpoints	Save progress at key steps so you can resume or rollback.
State capture	Back up current configuration and system state before making changes.
Rollback	Revert changes safely to a known good state.
Recovery after partial execution	Handle incomplete runs and finish safely.
Preserving forensic evidence	Keep logs, outputs and system state for investigation.

3. STOP CONDITIONS

- ✓ Define clear conditions to stop the automation.
- ✓ Check system health (CPU, memory, service status).
- ✓ Stop if error rate exceeds a threshold.
- ✓ Validate expected results before continuing.

Example: Stop if a service is unhealthy

```
#!/bin/bash
# Stop if service is not healthy
if ! systemctl is-active --quiet nginx; then
    echo "NGINX is not healthy. Stopping deployment."
    exit 1
fi
```

4. KILL SWITCHES

- ✓ Provide a manual way to stop the automation.
- ✓ Use a file-based or environment variable kill switch.
- ✓ Check the kill switch at safe points in the script.
- ✓ Ensure the process exits gracefully.

Example: File-based kill switch

```
#!/bin/bash
# Check for kill switch file
if [ -f /tmp/automation.stop ]; then
    echo "Kill switch detected. Stopping now."
    exit 1
fi
```

5. CHECKPOINTS AND STATE CAPTURE

- ✓ Save important state before making changes.
- ✓ Capture current configuration and system details.
- ✓ Store checkpoints in a safe location.
- ✓ Use checkpoints to resume or rollback.

Example: Create a checkpoint

```
#!/bin/bash
CHECKPOINT_DIR="/var/backups/automation"
mkdir -p $CHECKPOINT_DIR
cp -r /etc/nginx $CHECKPOINT_DIR/nginx_$(date +%F)
systemctl list-units --state=running > $CHECKPOINT_DIR/services_$(date +%F).txt
```

6. ROLLBACK STRATEGY

- ✓ Define how to revert changes before you start.
- ✓ Test rollback procedures regularly.
- ✓ Automate rollback where possible.
- ✓ Verify the system is back to a healthy state after rollback.

Example: Rollback configuration

```
#!/bin/bash
# Restore previous configuration
cp -r /var/backups/automation/nginx_2026-09-28/* /etc/nginx/
systemctl restart nginx
echo "Rollback completed. NGINX restored."
```

7. RECOVERY AFTER PARTIAL EXECUTION

- ✓ Detect what steps were completed.
- ✓ Resume from the last successful checkpoint.
- ✓ Clean up incomplete changes.
- ✓ Verify system health before continuing.

Example: Resume from last checkpoint

```
#!/bin/bash
LAST_STEP_FILE="/var/lib/automation/last_step"
# Read last successful step
LAST_STEP=$(cat $LAST_STEP_FILE)
echo "Resuming from step: $LAST_STEP"
```

8. PRESERVING FORENSIC EVIDENCE

- ✓ Keep detailed logs of all actions.
- ✓ Store command output and error messages.
- ✓ Do not delete temporary files immediately.
- ✓ Capture system state (configs, logs, process info).
- ✓ Protect logs from tampering.

Example: Save logs and system state

```
#!/bin/bash
LOG_DIR="/var/log/automation/$(date +%F)"
mkdir -p $LOG_DIR
exec >> (tee -a $LOG_DIR/run.log) 2>&1
echo "Starting automation at $(date)"
# Capture system info
date
uname -a
ps auxf > $LOG_DIR/processes.txt
```

9. KEY TAKEAWAYS

- ✓ Use stop conditions and kill switches to prevent bigger incidents.
- ✓ Take checkpoints and capture state before changes.
- ✓ Have a tested rollback strategy.
- ✓ Plan for recovery after partial execution.
- ✓ Preserve forensic evidence for investigation.

10. SENIOR ENGINEER TIP

Always test your incident response, rollback and recovery process in a staging environment before using it in production.

Capstone: Build a Production Linux Auto-Remediation System

detect | blast radius | evidence | validate | lock | remediate | verify | rollback | logs | metrics | preserve | escalate

1. PROJECT OVERVIEW



Build a production-ready Linux auto-remediation system that detects a degraded service, analyzes the impact, safely remediates the issue and verifies recovery. It also preserves evidence and escalates when automation cannot safely recover.

2. KEY LEARNING OUTCOMES

- ✓ Detect a degraded service.
- ✓ Establish blast radius.
- ✓ Gather and preserve evidence.
- ✓ Validate remediation conditions.
- ✓ Acquire an execution lock.
- ✓ Perform bounded remediation.
- ✓ Verify service recovery.
- ✓ Roll back when verification fails.
- ✓ Write structured logs.
- ✓ Emit operational metrics.
- ✓ Preserve incident evidence.
- ✓ Escalate when automation cannot safely recover.

3. TOOLS AND TECHNOLOGIES



Linux



Bash



systemd

metrics
(Prometheus)logs
(JSON)

4. PROJECT STRUCTURE

```
linux-auto-remediation/
├── scripts/      # detection, remediation, utils
├── configs/     # service definitions, thresholds
├── logs/        # structured logs
├── evidence/    # collected evidence (logs, configs)
├── metrics/     # metrics export files
├── systemd/    # service files and timers
└── README.md
```

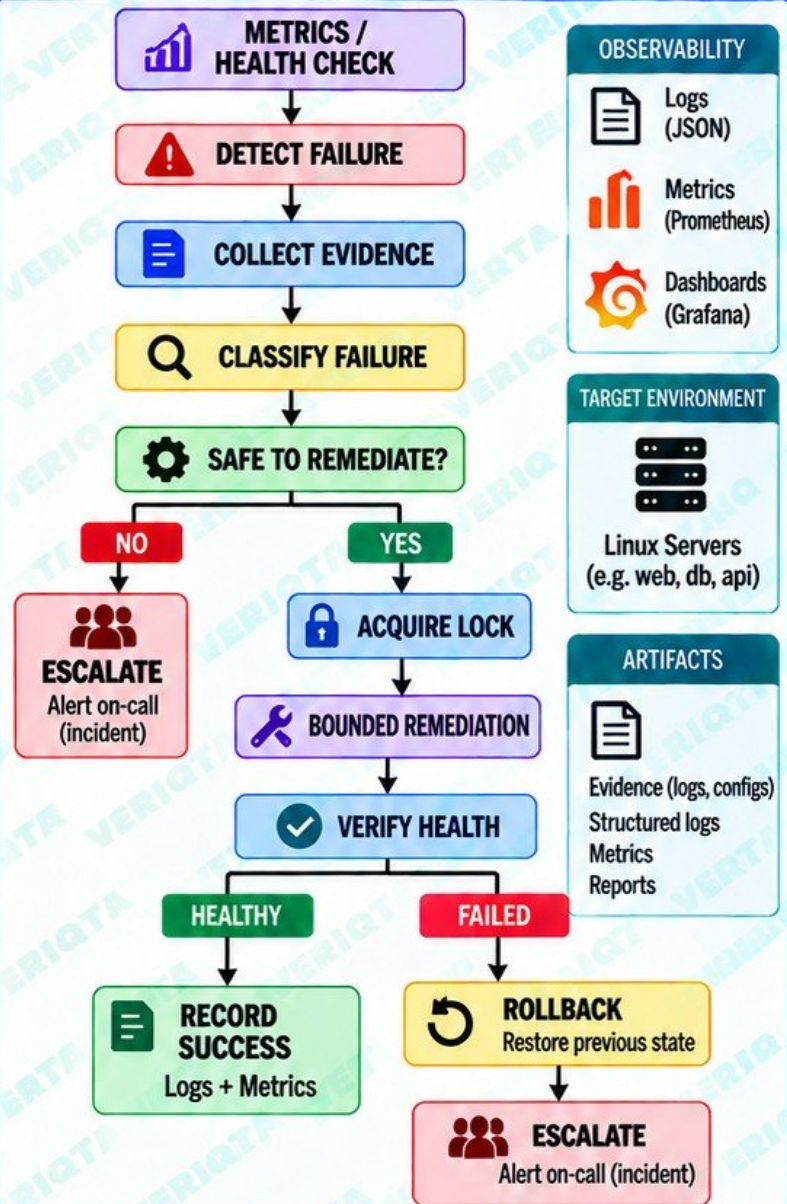
5. EXAMPLE: CHECK SERVICE HEALTH

```
#!/bin/bash
SERVICE="nginx"
if ! systemctl is-active --quiet $SERVICE; then
    echo "service_degraded=1"
    exit 1
else
    echo "service_degraded=0"
    exit 0
fi
```

6. EXAMPLE: WRITE STRUCTURED LOG

```
#!/bin/bash
echo '{"timestamp":"'$(date -Iseconds)', "event": "remediation",
"service": "nginx", "status": "success"}' >> logs/automation.log
```

7. HIGH-LEVEL ARCHITECTURE AND FLOW



8. VALIDATION CHECKS

- ✓ Confirm blast radius is within limits.
- ✓ Validate system state before remediation.
- ✓ Ensure lock is acquired.
- ✓ Verify service is healthy after remediation.
- ✓ Confirm metrics and logs are recorded.
- ✓ Test rollback and recovery.

9. PRESERVING INCIDENT EVIDENCE

- ✓ Store logs, system state and configs.
- ✓ Capture command output and timestamps.
- ✓ Keep evidence in a dedicated directory.
- ✓ Do not delete evidence after success.
- ✓ Use structured and tamper-resistant logs.

10. ESCALATION CONDITIONS

- ✓ Blast radius is too large.
- ✓ Remediation validation fails.
- ✓ Rollback also fails.
- ✓ Repeated failures (threshold exceeded).
- ✓ Unknown error or unsafe condition.
- ✓ Notify on-call with full context and evidence.

11. KEY TAKEAWAYS

- ✓ Automate with safety and control.
- ✓ Validate before making changes.
- ✓ Use locks to prevent concurrent runs.
- ✓ Keep changes small and bounded.
- ✓ Always verify and record results.
- ✓ Preserve evidence for investigation.
- ✓ Escalate when automation is not safe.